Operating Systems

Instructor

Dr.Eng. Mahmoud M. Ismail




Input Units
Output Units
Storage Units
CD/DVD
Processor
HD
Ram
Operating Systems



Input Units

Output Units

Storage Units



Introduction

- A computer is made up of hardware and software
- The hardware represents the body of the computer, that is, the tangible devices such as a screen, keyboard, processor, memory, disks and printers.
- But, It is not easy to use the hardware components of the computer without software.
- A computer without programs likes a body without a soul, or like a car without fuel.
- The most important of these programs is **the operating system**

➢ Therefore, the operating system is considered as the heart of the computer and its engine, and without it, the computer cannot be dealt with, run or used, and one of its most important functions is to manage hardware and provide a simple interface to user.

COURSE SPECIFICATIONS

Course Title: Operating Systems

Instructor: Dr. Mahmoud M. Isamail

Contact: dr.eng.mahmoudmmi@gmail.com

Course Description

- This course is an introduction to modern operating systems
- It presents the basic concepts, structure, design principles, implementations issues and mechanisms of operating systems.
- It provides the frame of reference of how an operating systems work and how they are designed in terms of their efficiency and reliability.

Course objectives

1) Providing an overview of computer system hardware
2) Understanding the basic concepts of operating systems
3) Knowing the components and objectives of the operating system
4) Highlighting the importance of operating systems by identifying the services of operating systems.
5) Learning how operating systems have evolved from Simple Batch Systems to Multiprogrammed, Time Sharing, Multiprocessing and Distributed.
6) Knowing Major Achievements & Characteristics of modern OS
7) Understanding how does an operating system manage the processes (program in execution) and the resources in a convenient and efficient way.
8) Knowing the types of dual mode in operating system and Why?
9) Understanding the distinction between process and thread, the difference between user-level threads and kernel-level threads as well as describe the basic design issues for threads.

Course objectives

10) Controlling concurrent access to shared data by several processes so that data consistency is maintained (Mutual Exclusion and Synchronization).

11) Define the different ways of memory management (memory management techniques) to allocate it to processes and the advantages and disadvantages of each.

12) Describe the hardware and control structures that support virtual memory

13) Describe the various OS mechanisms used to implement virtual memory

14) Learning CPU scheduling algorithms for processes and scheduling criteria

15) Understanding the basic concepts of files, the principal techniques for file organization and access and file management systems of operating systems.

16) Understanding the disk scheduling algorithms to handle a number of I/O requests (reads and writes) from various processes in the queue.

Intended learning outcomes of course (ILOs)

By the completion of the course the students should be able to:

1) **Explain the objectives and functions of operating systems**
2) **Describe the various types of operating system services**
3) **Describe how operating systems have evolved from Simple Batch Systems to Multiprogrammed, Time Sharing, Multiprocessing and Distributed.**
4) **Identify the characteristics of modern OS**
5) **Differentiate among process states**
6) **Explain the objective of dual mode in operating system**
7) **Compare between user and kernel mode**
8) **Compare and contrast between process and thread, user-level threads and kernel-level threads.**
9) **Explain the major benefits and significant challenges of designing multithreaded processes.**

Intended learning outcomes of course (ILOs)

By the completion of the course the students should be able to:

10) **Identify the basic concepts related to concurrency, such as(race condition, mutual exclusion and critical section).**
11) **Classify/categorize the solutions to the critical section problem (Hardware, Software and Operating system).**
12) **Compare and contrast the different ways of memory management and the advantages and disadvantages of each.**
13) **Identify the difference between physical and logical addresses**
14) **Discuss Memory Partitioning (Fixed and Dynamic partitioning)**
15) **Explain the concept of paging and segmentation**
16) **Assess the relative advantages of paging and segmentation**
17) **Identify hardware and software support for virtual memory management**
18) **Identify the objective of processes scheduling**

Intended learning outcomes of course (ILOs)

19) Differentiate among types of scheduler such as short-term, medium-term, and long-term.

20) Compare and contrast CPU scheduling algorithms used for both preemptive and non-preemptive scheduling of processes such as FCFS, SJF, priority and Round Robin.

21) Compare and contrast the file organizations techniques (sequential file, indexed sequential file, indexed file, direct, or hashed file).

22) Identify the file Access rights (Execution, Reading, Appending, Updating ,Changing protection and Deletion)

23) Compare and contrast the disk scheduling algorithms to handle I/O requests

- Selection according to requestor (Random-FIFO- PRI- LIFO)
- Selection according to requested item (SSTF-SCAN-C-SCAN)

COURSE CHAPTERS

Chapter 1: Computer System Overview

Chapter 2: Operating System Overview

Chapter 3: Process Description and Control (Processes Management)

Chapter 4:Threads

Chapter 5: Concurrency: Mutual Exclusion and Synchronization

Chapter 6: Memory Management

Chapter 7: Virtual Memory

Chapter 8: Uniprocessor Scheduling

Chapter 9: File Management & Disk Scheduling

Teaching and learning methods

- Lectures
- Seminars
- Discussion Groups

Student Assessment Methods

- Exams to assess knowledge and understanding.
- Assignments to assess intellectual, professional & practical skills.
- Final exam to assess knowledge and understanding.

Weighting of assessments

- 7th Exam + Section — 30 marks (25/5)
- 12th Exam + Section — 20 marks (15/5)
- Lab (Quizzes & Assignments) — 10 marks
- Final Exam — 40 marks

Textbook:

STALLINGS, W., Operating Systems: Internals & Design Principles, 9th Edition, Pearson, 2018.

References:

Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, Operating Systems Concepts, 10th Edition, Wiley, 2018.

Operating Systems
Chapter 1
Computer System Overview

Instructor

Dr.Eng. Mahmoud M. Ismail

Contact: dr.eng.mahmoudmmi@gmail.com
dr.mahmoudmmi@yahoo.com

Outline

- **Basic Elements of a computer system**
- **Processor registers**
- **Instruction Execution**
- **Interrupts**
  - Classes of Interrupts
  - Interrupt Handler
  - Instruction Cycle with Interrupts
  - Interrupt Processing
  - Multiple Interrupts and approaches dealing with multiple interrupts
- **Memory Hierarchy**
- **Cache Memory**
- **I/O Communication Techniques**
  - Programmed I/O
  - Interrupt-Driven I/O
  - Direct Memory Access (DMA)

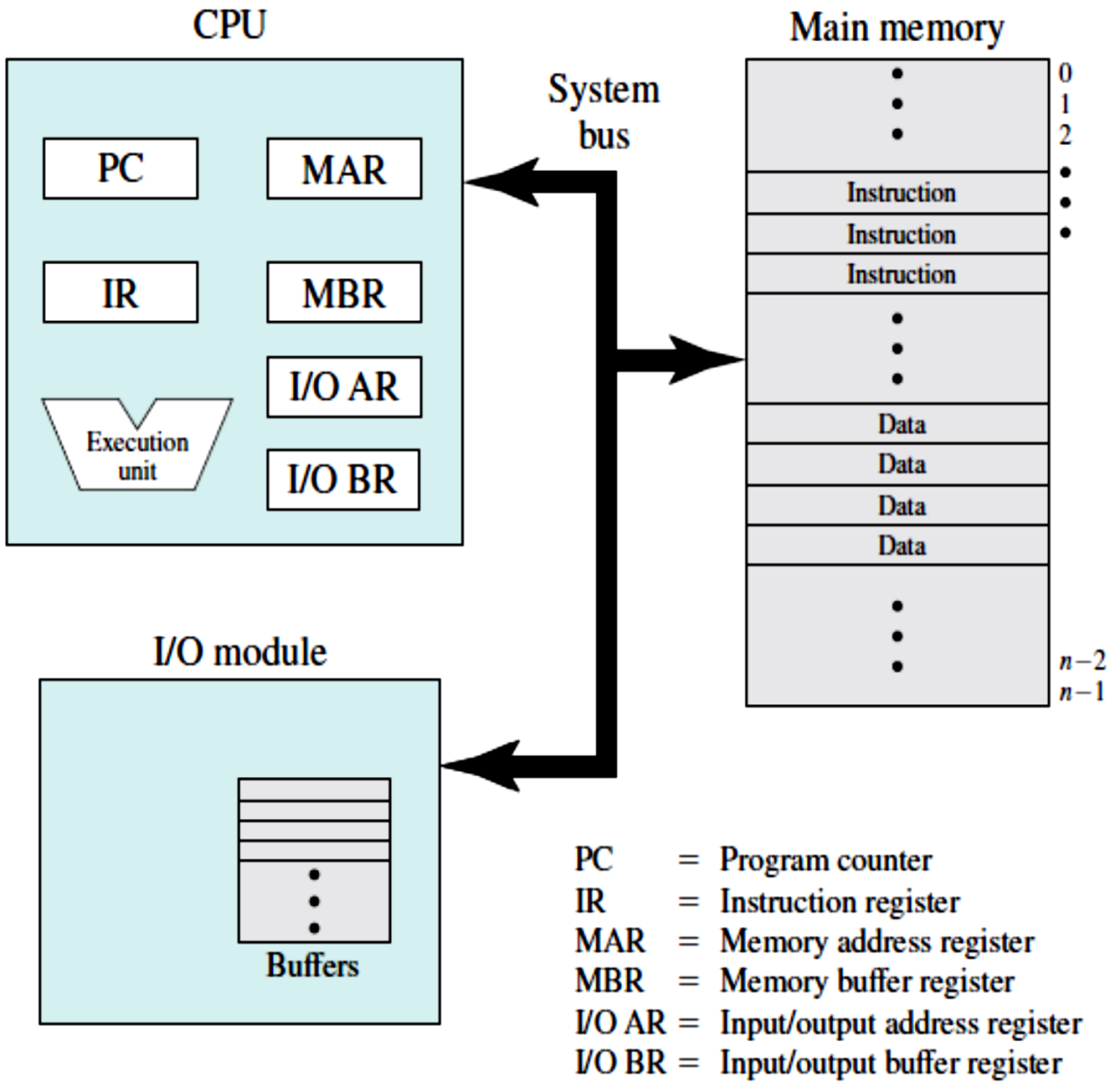
Operating System

- OS exploits the hardware resources of one or more processors to provide a set of services to users.
- OS also manages main and secondary memory and I/O devices on behalf of its users.

✓ **Therefore, it is important to have some understanding of the underlying computer system hardware**

Basic Elements

- **Processor**
- **Main Memory**
- **I/O modules**
- **System bus**



Processor

Controls the operation of the computer

Performs the data processing functions

Referred to as the Central Processing Unit (CPU)

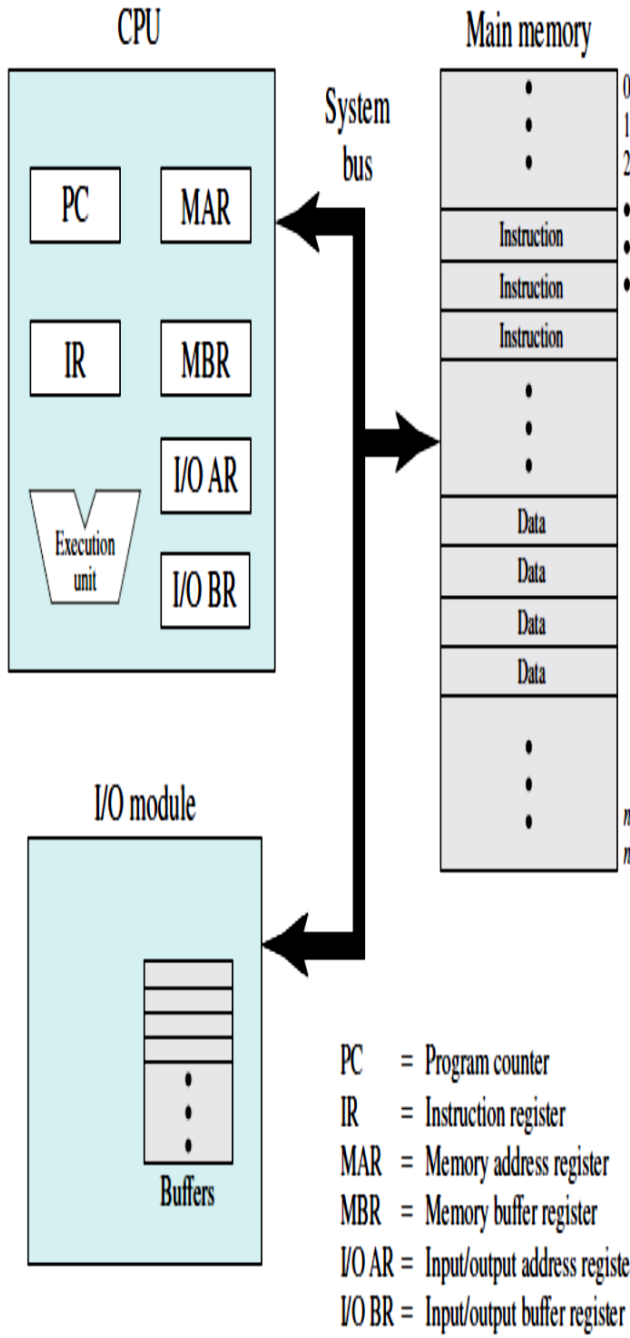
Main Memory

- Stores **data** and **programs**
- **Volatile** (Contents of the memory is lost when the computer is shut down)
- Referred to as **real memory** or **primary memory**

I/O Modules

Moves **data** between **computer** and **external environments** such as:

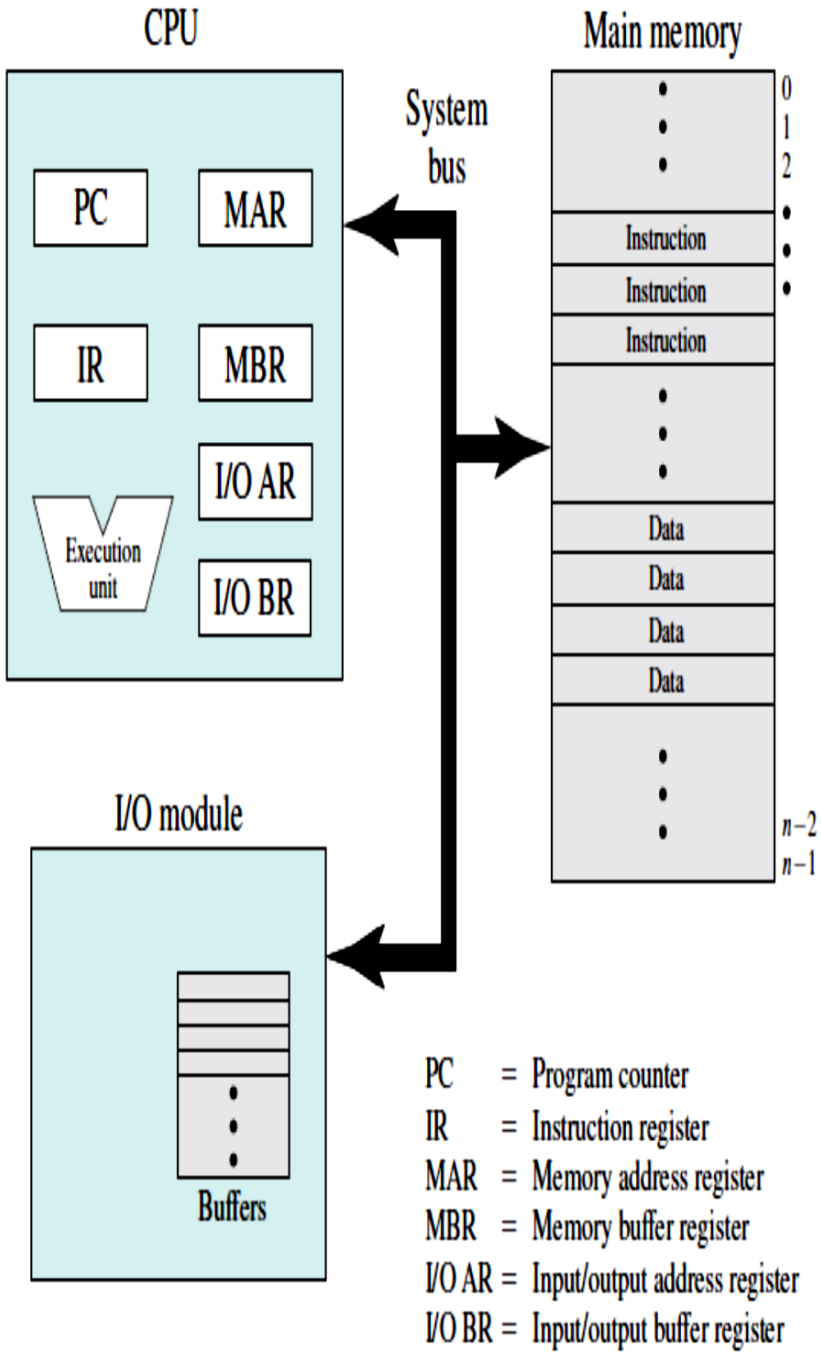
- **storage** (e.g. hard drive)
- **communications equipment**
- **terminals**



- Transfers **data** from **external devices** to **processor** and **memory**, and vice versa.
- It contains **internal buffers** for temporarily storing data until they can be sent.

System Bus

- Provides **communication** among processors, main memory, and I/O modules



Program Counter
PC

Contains **the address of an instruction** to be fetched

Instruction Register
IR

Contains **the instruction** most recently fetched

Memory Address Register
MAR

Specifies **the address in memory** for **the next read** or **write**

Memory Buffer Register MBR

Contains **data** to be **written or read** into/from memory

I/O address register (I/OAR)

Specifies **a particular I/O device**

I/O buffer register (I/OBR)

Used for **the exchange of data** between **an I/O module** and the **processor**

Processor Registers

1) **User-visible registers**
   - Enable **programmer** to **minimize main-memory references** by **optimizing register use**
2) **Control and status registers**
   - Used by **processor** to **control operating of the processor**
   - Used by **operating-system routines** to **control the execution of programs**

User-Visible Registers

- May be referenced by machine language that the processor executes while in **user mode.**
- Available to all programs (application programs and system programs)
- Types of registers
  - Data
  - Address
    - Index
    - Segment pointer
    - Stack pointer

User-Visible Registers

- Address Registers
  - Index
    - involves adding an index to a base value to get an address
    - It is useful for stepping through arrays.
      It can also be used for holding loop iterations and counters.
  - Segment pointer
    - when memory is divided into segments, memory is referenced by a segment and an offset within the segment
  - Stack pointer
    - points to top of stack

Control and Status Registers

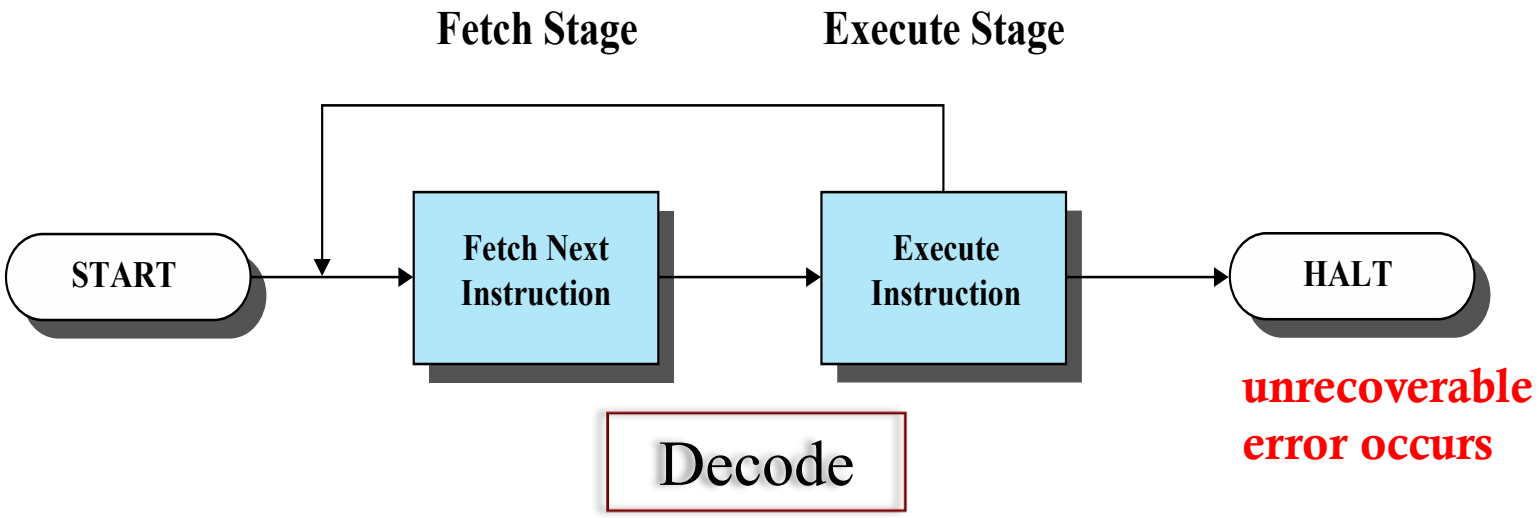
- Program Counter (PC)
  - Contains the address of an instruction to be fetched
- Instruction Register (IR)
  - Contains the instruction most recently fetched
- Program Status Word (PSW)
  - condition codes
  - Interrupt enable/disable
  - Supervisor/user mode

Control and Status Registers

- Condition Codes or Flags
  - Bits set by the processor hardware as **a result of operations**
  - Can be accessed by a program but not altered
  - Examples
    - positive result
    - negative result
    - zero
    - Overflow

Instruction Execution Cycle

- A program consists of a set of instructions stored in memory

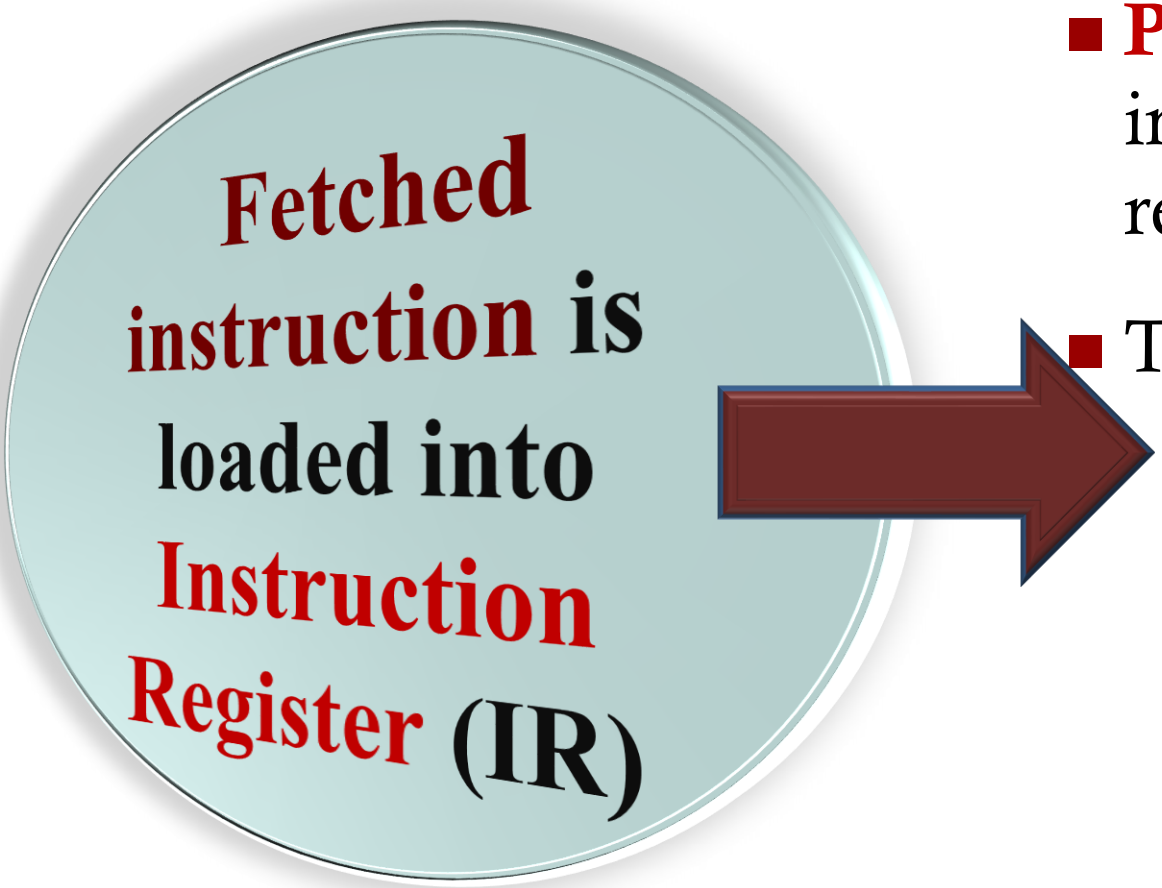


Basic Instruction Cycle

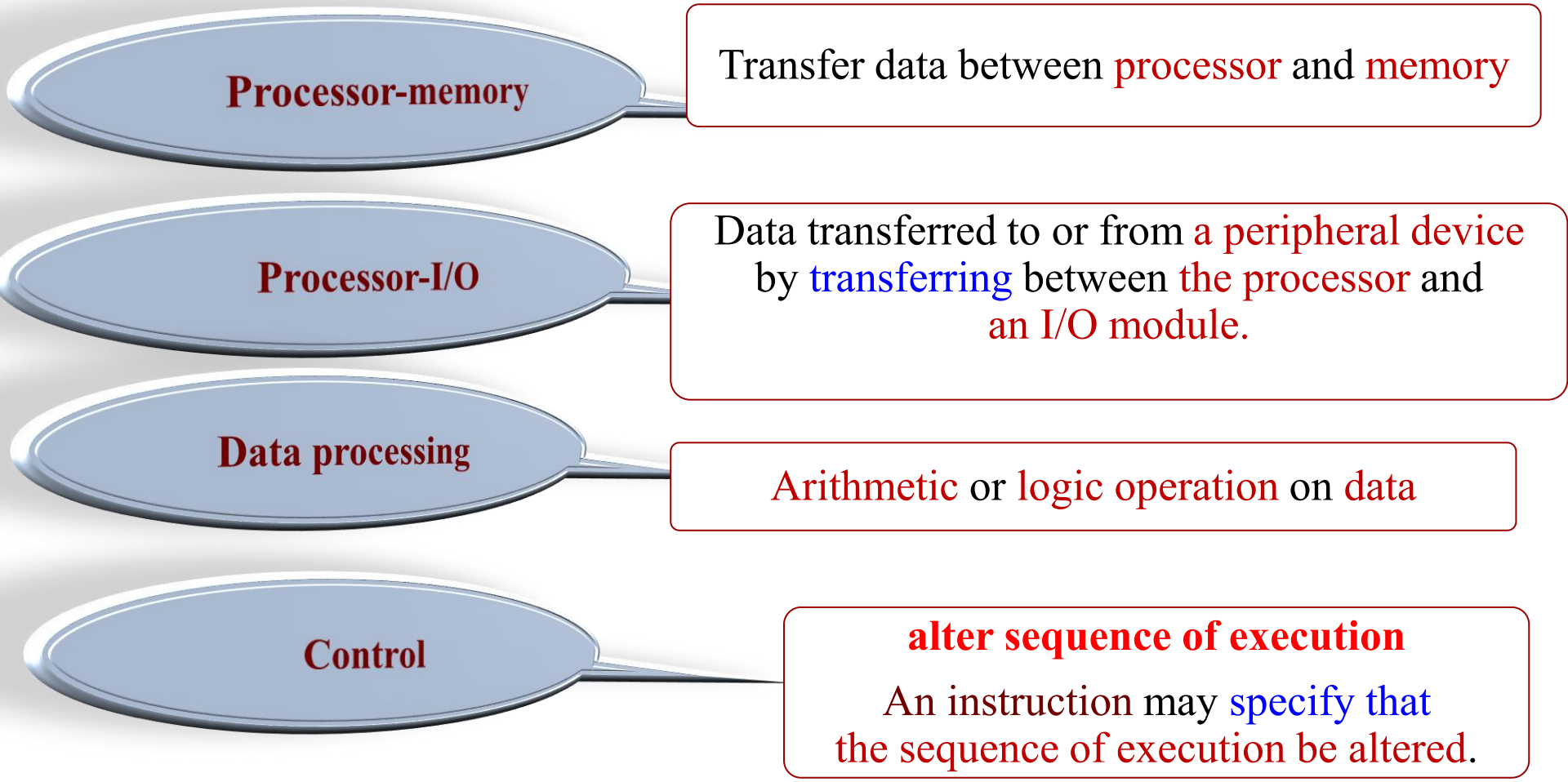
Instruction Fetch and Execute

- The processor fetches the instruction from memory
- Program counter (PC) holds address of the instruction to be fetched next.
  - PC is **incremented after each fetch** so, it will fetch the next instruction in sequence.

Instruction Register (IR)



- **Processor** interprets the instruction and performs required action.
- Types of instructions:
  - Processor-memory
  - Processor-I/O
  - Data processing
  - Control



- For example, the processor may fetch an instruction from location **149**, which specifies that the next instruction be from location **182**.
- The processor sets the program counter to **182**.
- Thus, on the next fetch stage, the instruction will be fetched from location **182** rather than **150**.

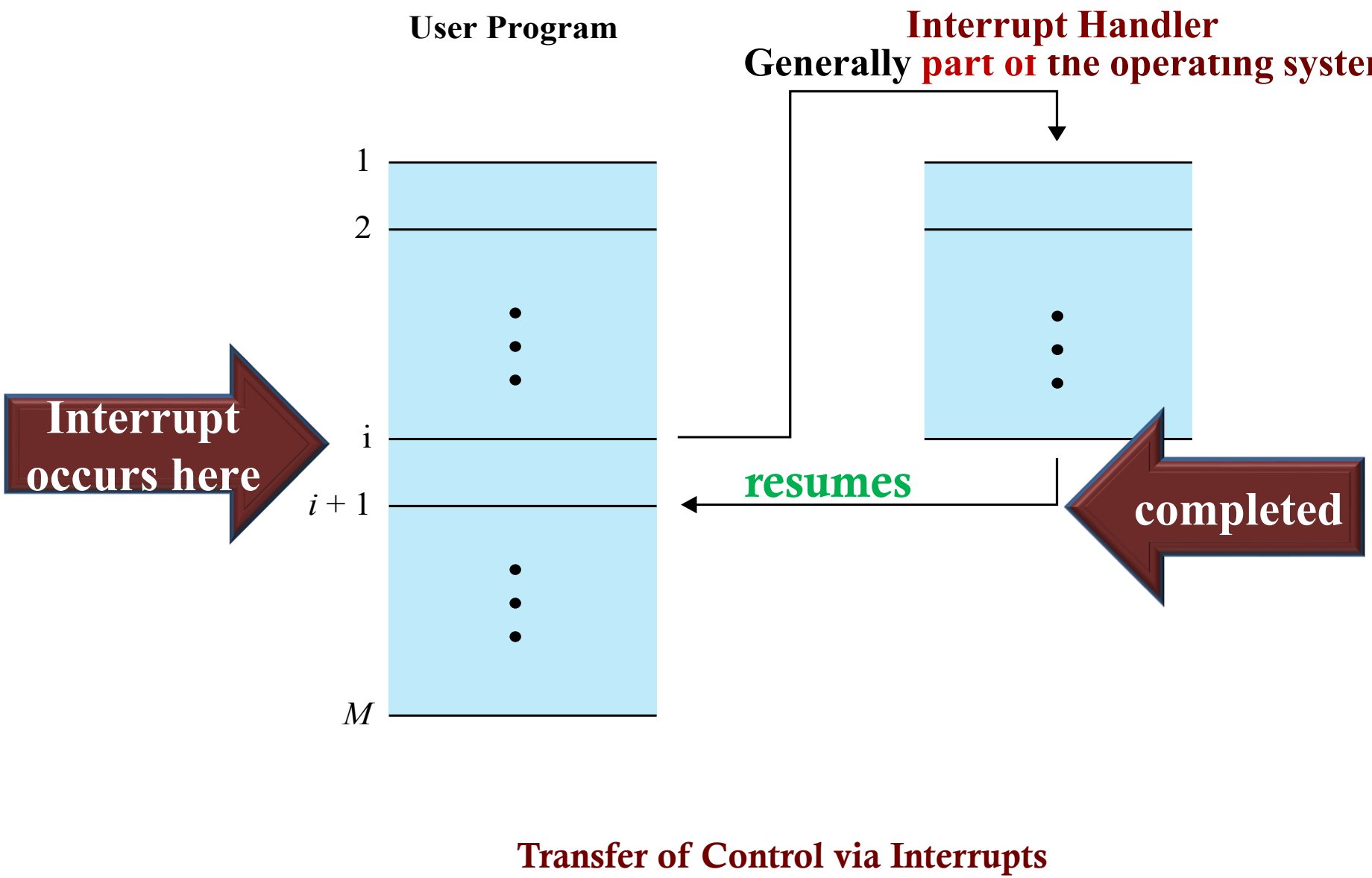
Interrupts

- **An interruption** of **the normal sequence of execution**

- **An interrupt is a signal sent to processor emitted by hardware or software indicating that an event needs attention.**

- **The processor responds by suspending its current activities, saving its state, and executing a function called an interrupt handler (part of the OS) to deal with the event.**

- **This interruption is temporary, and, after the interrupt handler finishes, the processor resumes normal activities.**



Transfer of Control via Interrupts

Interrupts

- Provided to **improve processor utilization Why???**
  - Most **I/O devices** are **slower** than **the processor** (processor is transferring data to a printer-hard disk)
  - **Processor** must **pause to wait** for device
  - **Wasteful use of the processor**

✓ Interrupts allows the processor to execute other instructions while an **I/O operation** is in progress

Classes of Interrupts

S/W	
Generated by **Applications** due to some condition that occurs as a result of an instruction execution, such as **arithmetic overflow**, **division by zero**, attempt to execute an **illegal machine instruction**, and **reference outside a user's allowed memory space.**	
H/W	
I/O	Generated by an I/O controller, to **signal normal completion of an operation** or to signal a variety of **error conditions**.
Hardware failure	Generated by a failure, such as **power failure** or **memory parity error**.
Timer	Generated by **a timer** within **the processor**. This allows **the operating system** to perform certain functions on a regular basis.

Memory parity error:

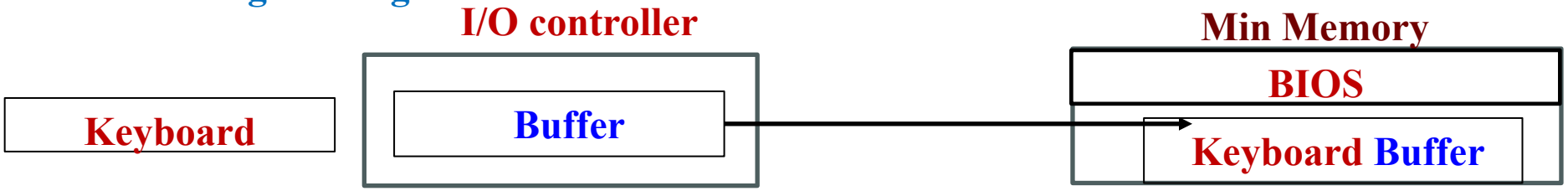
Electrical or magnetic interference from internal or external causes can cause a single bit of memory to spontaneously flip to the opposite state.

•Overheating or cooling.

Hardware Interrupt

Used by devices to communicate that they require attention from OS.

- When the external device becomes ready to be serviced (that is, when it is ready to accept more data from the processor) the I/O module for that external device sends an interrupt request signal to the processor.
- The processor responds by suspending operation of the current program; branching off to a routine to service that particular I/O device known as an interrupt handler (Generally part of the operating system) and resuming the original execution after the device is serviced.



- When a user typed on the keyboard the typed data is stored in the buffer.
- IO controller generates an interrupt to inform the processor that the data is available in the buffer.
- For example, pressing a key on a computer keyboard or moving the mouse, triggers interrupts that cause the processor to read the keystroke by calling interrupt handler which read the key, or the mouse's position, and copy the associated information into the computer's memory.

Hardware Interrupt

- For the user program, an interrupt suspends the normal sequence of execution.
- When the interrupt processing is completed, execution resumes.
- Thus, the user program does not have to contain any special code to accommodate interrupts.

The processor and the OS are responsible for suspending the user program, then resuming it at the same point.

Interrupt Handler (part of the OS)

- **A program** that determines **nature of the interrupt** and **performs whatever actions are needed**.
- Control is transferred to **this program**
- Generally part of **the operating system**



S/W Interrupt
Handled by OS

H/W Interrupt
Handled by BIOS

BIOS(Basic Input Output System)

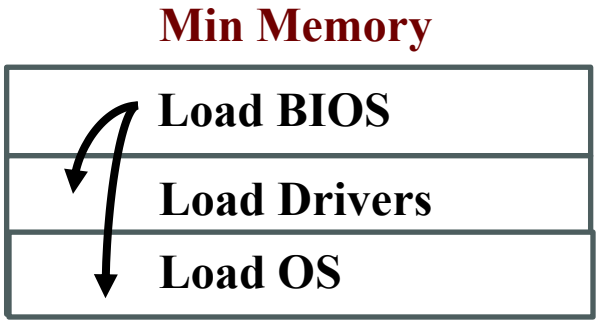
- **BIOS, computer program that is typically stored in EPROM before on ROM and used by the CPU to perform start-up procedures when the computer is turned on.**

❖ **Any H/W has a device driver loaded into a certain address in memory by BIOS**

- **The first program loaded into memory is BIOS why ?**
  Because it is the database of H/W (database of drivers) so, it knows where the device driver of H/W in memory.

➢ **Its two major procedures are:**

1) **Determining what peripheral devices (keyboard, mouse, disk drives, printers, video cards, etc.) are available**
2) **Loading OS into main memory.**



When users turn on their computer, the microprocessor passes control to the BIOS program, which is always located at the same place on EPROM.

➢ **So, neither the OS nor the application programs need to know the details of the peripherals (such as hardware addresses).**

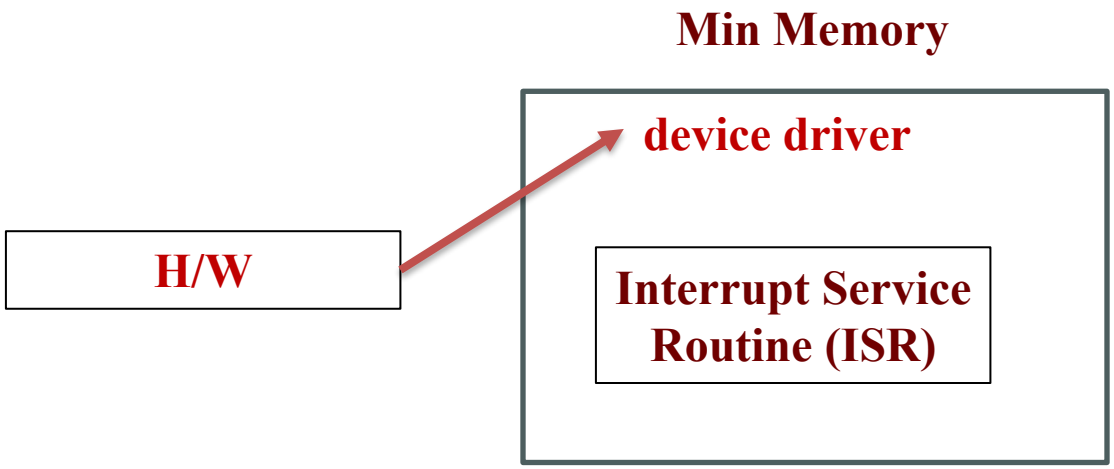
The functions of BIOS

BIOS identifies, configures, tests and connects computer hardware to the OS immediately after a computer is turned on. The combination of these steps is called the boot process.

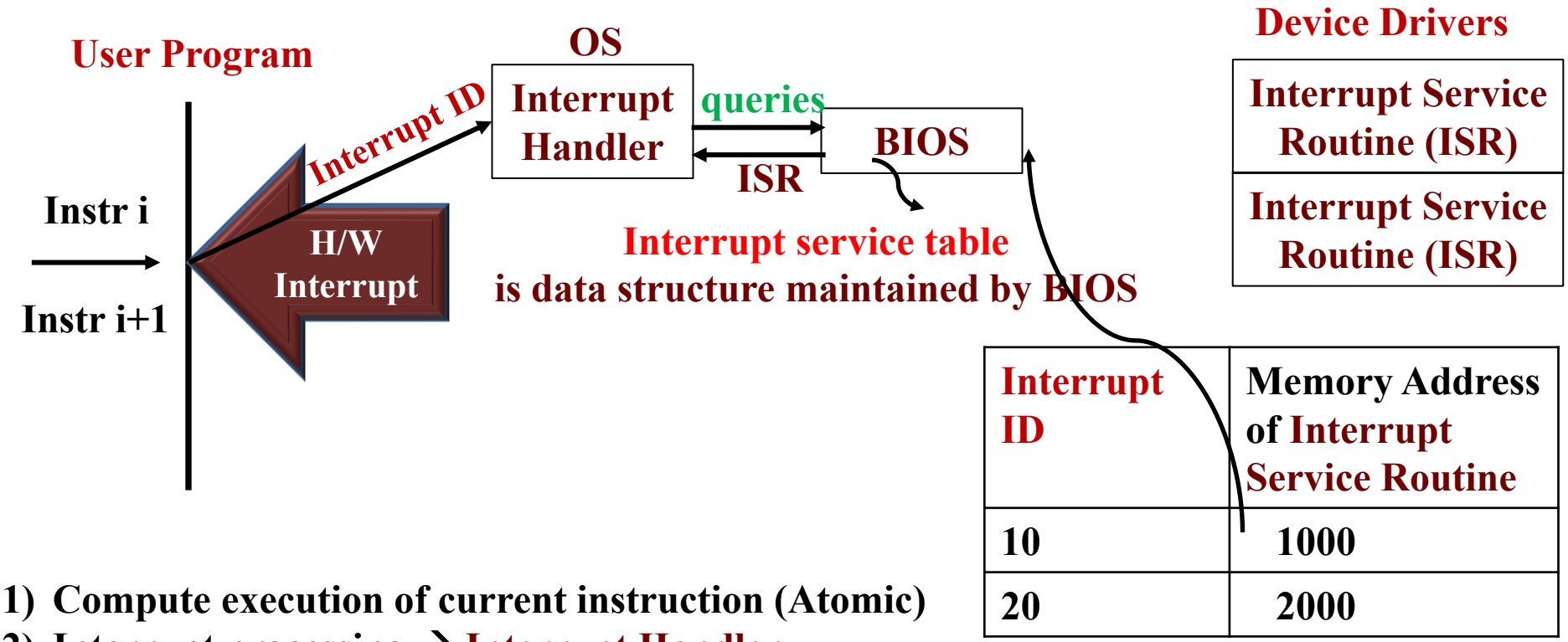
1) **Power-on self-test (POST): tests the hardware of the computer before loading the OS.**
2) **Software/drivers: locates the software and drivers that interface with the OS once running.**
3) **Bootstrap loader: locates the OS.**

Processing of H/W Interrupt

- Any H/W has a device driver loaded into a certain address in memory
- The device driver contains a function (Interrupt Service Routine ISR) to deal with certain interrupt.



Processing of H/W Interrupt



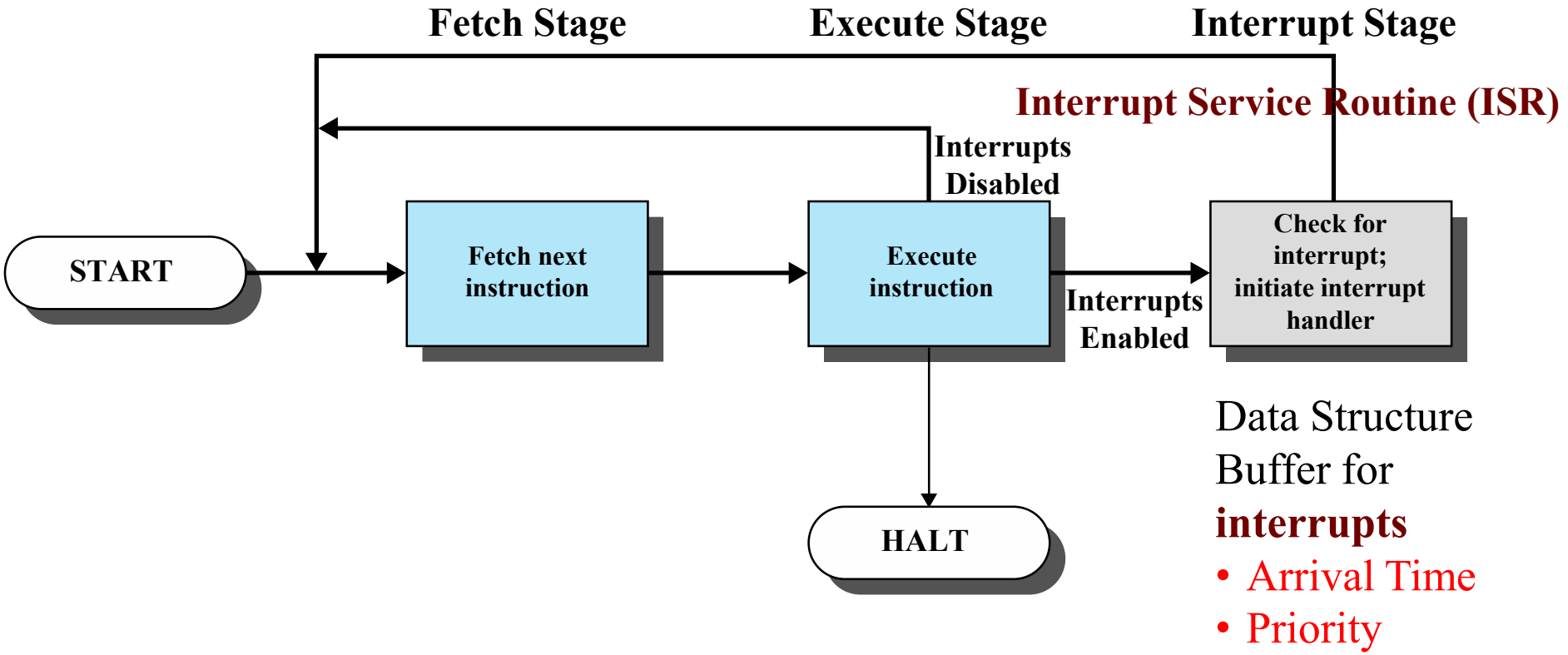
Interrupt ID	Memory Address of Interrupt Service Routine
10	1000
20	2000

1) Compute execution of current instruction (Atomic)
2) Interrupt processing → Interrupt Handler

- Interrupt sent to Interrupt Handler that classify interrupt
- OS (Interrupt Handler) queries BIOS to consult Interrupt service table to get ISR address
- ISR address returned to Interrupt handler.
- The ISR is executed and then the execution of program is resumed.

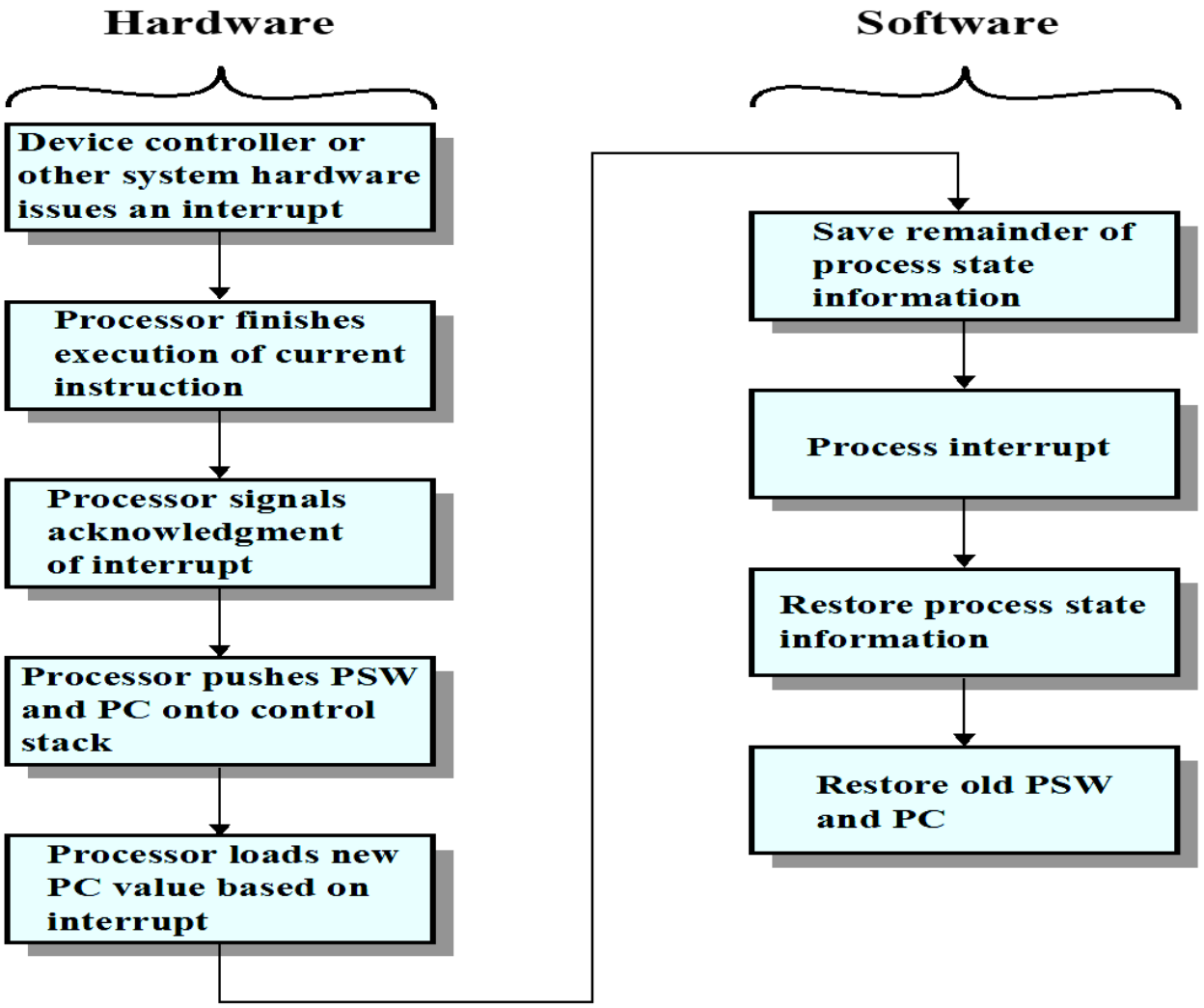
Before handling interrupt PC is saved (to resume execution of program)

Instruction Cycle with Interrupts



- **Processor** checks for **interrupts**
- If **no interrupts**, fetch **the next instruction** for the current program
- If **an interrupt** is **pending**, **suspend execution of the current program**, and **execute** the **interrupt handler**.
- When **the interrupt-handler routine** is **completed**, **the processor** can **resume** execution of the user program at the point of interruption.

Interrupt Processing



Simple Interrupt Processing

Interrupt Processing

An interrupt triggers a number of events, both in the processor hardware and in software. When an I/O device completes an I/O operation, the following sequence of hardware events occurs:

1) **The device** issues **an interrupt signal** to **the processor**.
2) **Processor** finishes execution of the current instruction before responding to the interrupt.
3) **Processor** checks for **an interrupt**. If there is one, it then sends **an acknowledgement** signal to **the I/O device** that issued the interrupt. This signal allows **the device to** remove **its interrupt signal**.
4) **The processor** next needs to prepare to transfer control to **the interrupt routine** and **saves information** needed **to resume the current program** at the point of interrupt.

The minimum information required is **the location of the next instruction to be executed** (**PC**) and **program status word** (PSW).

These can be pushed onto a control **stack**.

5) **Processor** then loads **PC** with the entry location of the interrupt-handling routine that will respond to this interrupt.

Interrupt Processing

The control is transferred to the **interrupt-handler program**. The execution of this program results in the following operations:

6) **The contents of the processor registers** need to **be saved**, **because these registers** may be used by the **interrupt handler**. Typically, **the interrupt handler** will begin by saving the contents of all registers on the **stack**.

7) The **handler** performs the **interrupt processing**.

8) **When interrupt processing** is **complete**, **the saved register values** are **retrieved** from the **stack** and restored to the registers).

9) **Finally**, is to restore the old **PSW** and **program counter values** from the stack.
As a result, **the next instruction** to be executed will be from the previously interrupted program.

Program status word (PSW)

- All processor designs include a register or set of registers, often known as the **program status word** (PSW), that contains **condition codes** plus other **status information**.

- **Condition codes**: Result of the most recent arithmetic or logical operation (e.g., positive result , negative result , zero, overflow).
- **Status information**: Includes interrupt enable/disable bit , execution mode (kernel/user-mode bit).

Multiple Interrupts

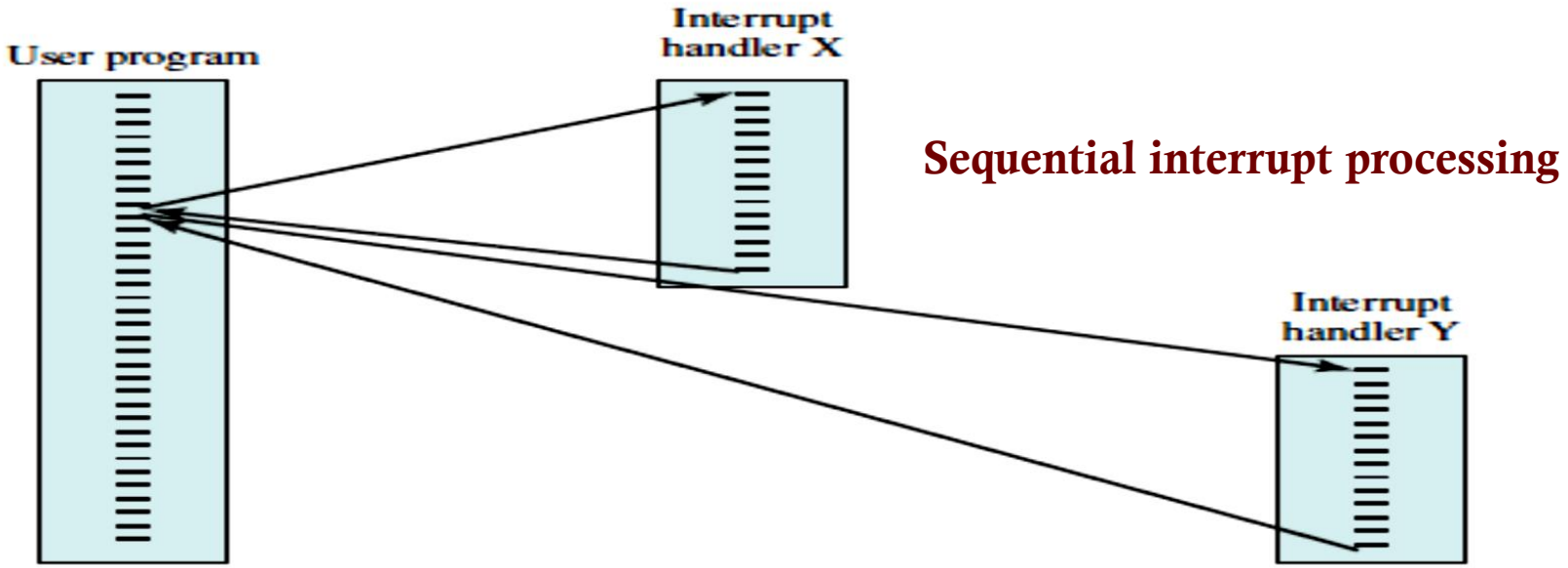
An interrupt occurs while another interrupt is being processed

- e.g. a program may be receiving data from communications line and printing results **at the same time**

- The **printer** will generate **an interrupt** every time that **it completes a print operation**.
- The **communication line controller** will generate **an interrupt** every time **a unit of data arrives**.
- It is possible for **a communications interrupt** to **occur** **while** **a printer interrupt** is being **processed**.

Approaches dealing with multiple interrupts
Transfer of Control with Multiple Interrupts

1) Disable interrupts while an interrupt is being processed



Sequential interrupt processing

- **The processor ignores** any **new interrupt** while processing **one interrupt**.
- If **an interrupt** occurs during this time, it remains **pending/disabled** and will be checked by the processor after the processor has reenabled interrupts.
- After **the interrupt-handler routine** completes, **interrupts** are **reenabled** before resuming the user program, and **the processor** checks to see if additional **interrupts** have occurred.

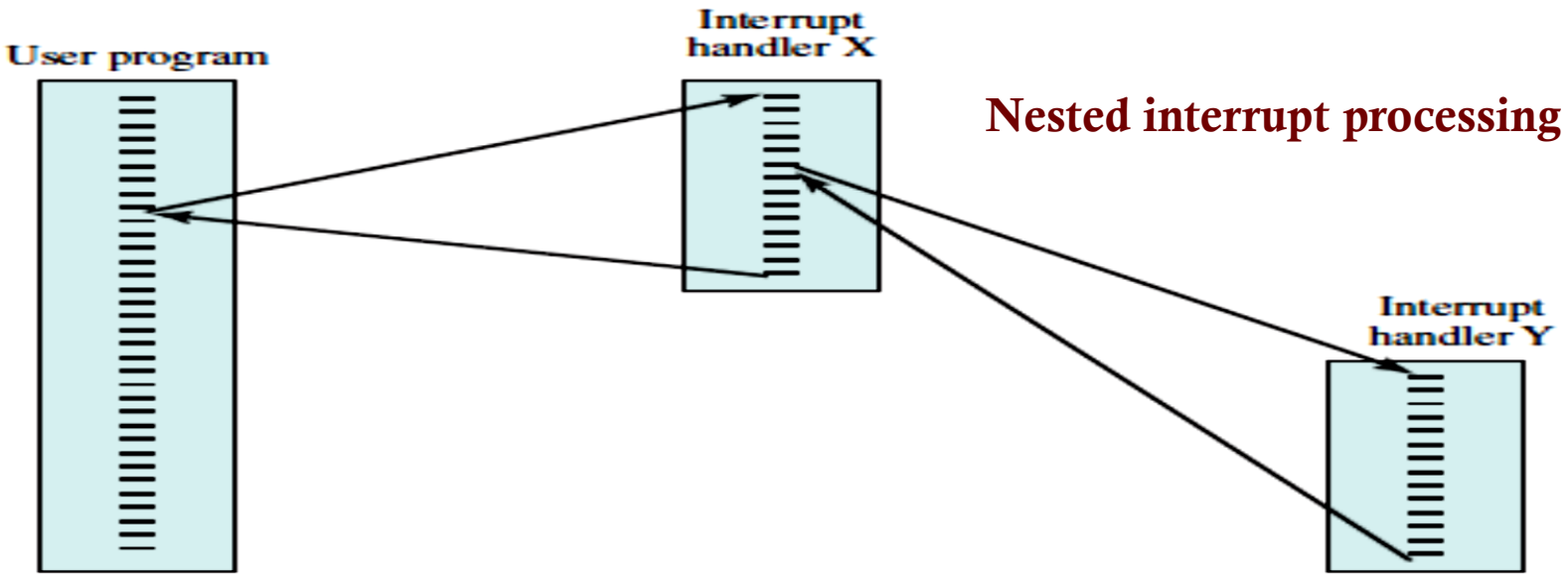
The drawback of the approach

It does not take into account relative **priority** or **time-critical needs**.

For example, when **input arrives from the communications line**, it may need to be absorbed rapidly to make room for more input.

2) Define priorities for interrupts

Allow an interrupt of **higher priority** to cause a lower–priority interrupt handler to be interrupted/wait.



For example, when **input** arrives from the **communications line**, **it needs to be absorbed quickly to make room for more input**.

Multiprogramming

- Processor has more than one program to execute
- The sequence the programs are executed depend on their **relative priority** and whether they are waiting for I/O.
- After **an interrupt handler completes**, **control** may not return to the program that was executing at the time of **the interrupt**.

The use of the stack in interrupt handling

- The stack is used to save the state of the CPU including the program counter and status register, allowing the interrupt service routine to execute.
- After handling the interrupt, the original state is restored from the stack to continue program execution.

Also, The stack is useful in various programming scenarios such as function call management.

Note:

1) In case of more than one interrupt → interrupt buffer
2) Interrupt ca be organized on Arrival Time or Priority
3) Multiple cores can handle independent interrupt

Memory Hierarchy

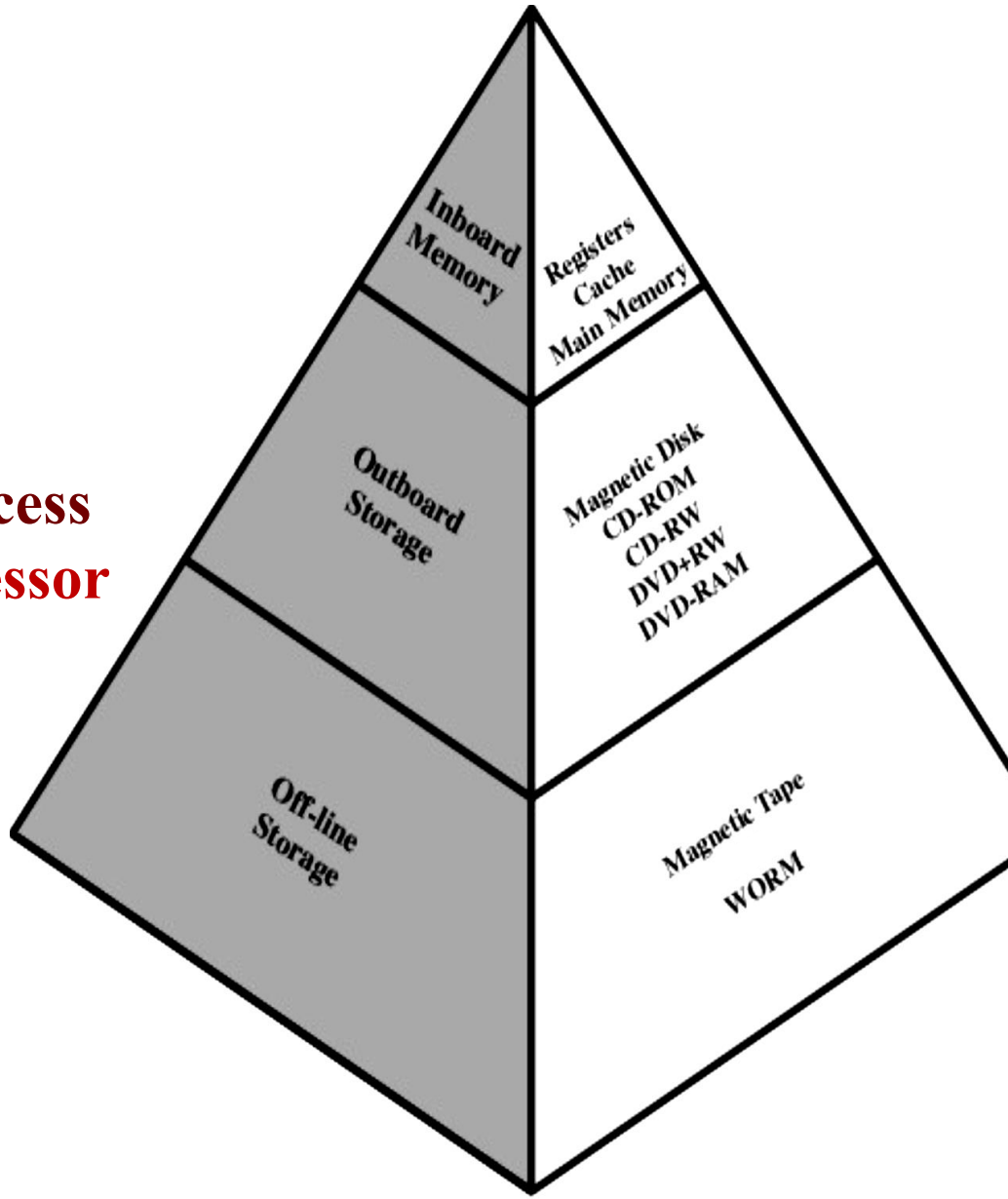
- **Major constraints in memory**
  - **amount**
  - **speed**
  - **expense**
- **Memory must be able to keep up with the processor**

  Processor speed is faster than memory speed

  - **That is, as the processor is executing instructions, we would not want it to have to pause waiting for instructions or operands.**
- **Cost of memory must be reasonable in relationship to the other components**

The Memory Hierarchy

- Going down the hierarchy:
  - Decreasing cost per bit
  - Increasing capacity
  - Increasing access time
  - Decreasing frequency of access to the memory by the processor



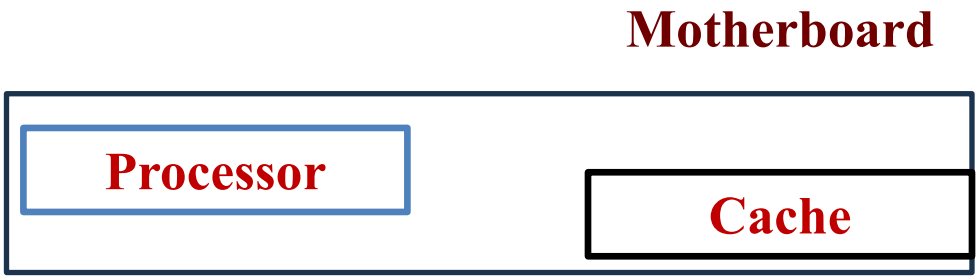
Cache Memory: Why?

- **Processor must access memory at least once per instruction cycle to fetch the instruction, and often one or more additional times, to fetch operands and/or store results.**
- **Processor execution is limited by memory cycle time (the time it takes to read one word from or write one word to memory).**

Processor speed is faster than memory speed

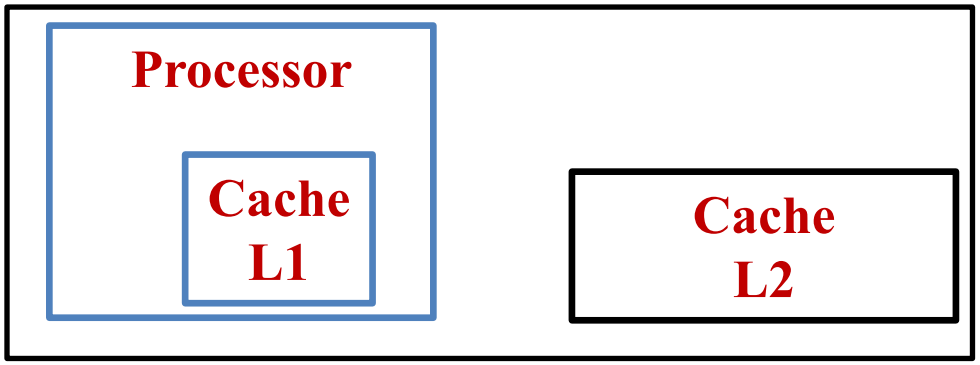
➢ **The solution is providing a small, fast memory between the processor and main memory, namely the cache (Invisible to OS).**

Types of Cache Memory



1) **L1 cache**

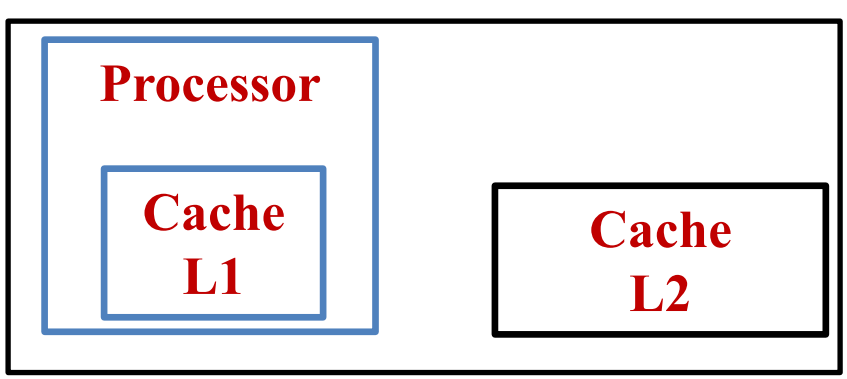
- Sometimes called **Primary Cache**, is **the smallest** and **fastest memory** level.
- It is commonly **64KB** in size and up to **100 times faster than RAM**.
- If a CPU has four cores (quad core CPU), then each core will have its own level 1 cache.
- So, a quad-core CPU would have a total of 256 KB
- If the processor fails to find the required data in **L1**, it looks for it in the **L2** and **L3 cache**.

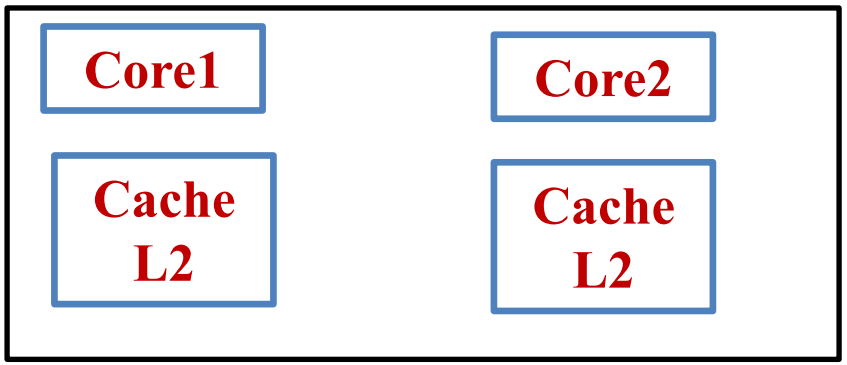


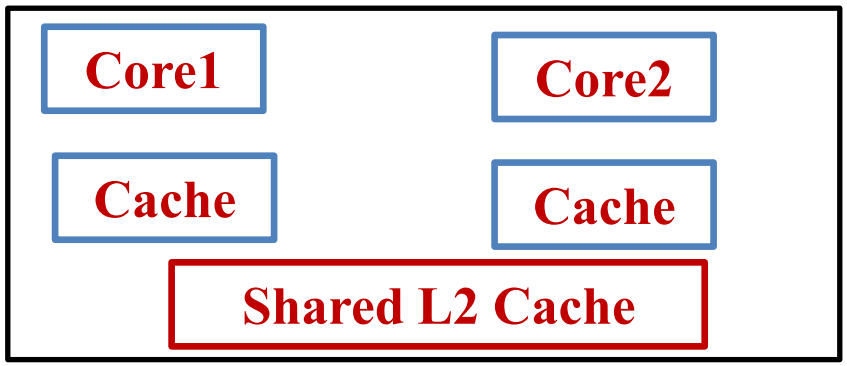
2) **L2 cache. This level 2 cache may be :**

- **Outside the CPU (on a separate chip) and is placed between the primary cache and the rest of the memory.**
- **Or Inside the CPU**
  All the cores of a CPU can have their own separate level 2 cache or they can share one L2 cache among themselves.

- **The memory size of this cache is in the range of 256 KB to the 512 KB, sometimes as high as 1MB. (256KB to 32MB).**
- **It is slower than the L1 cache. (about 25 times as fast as RAM).**

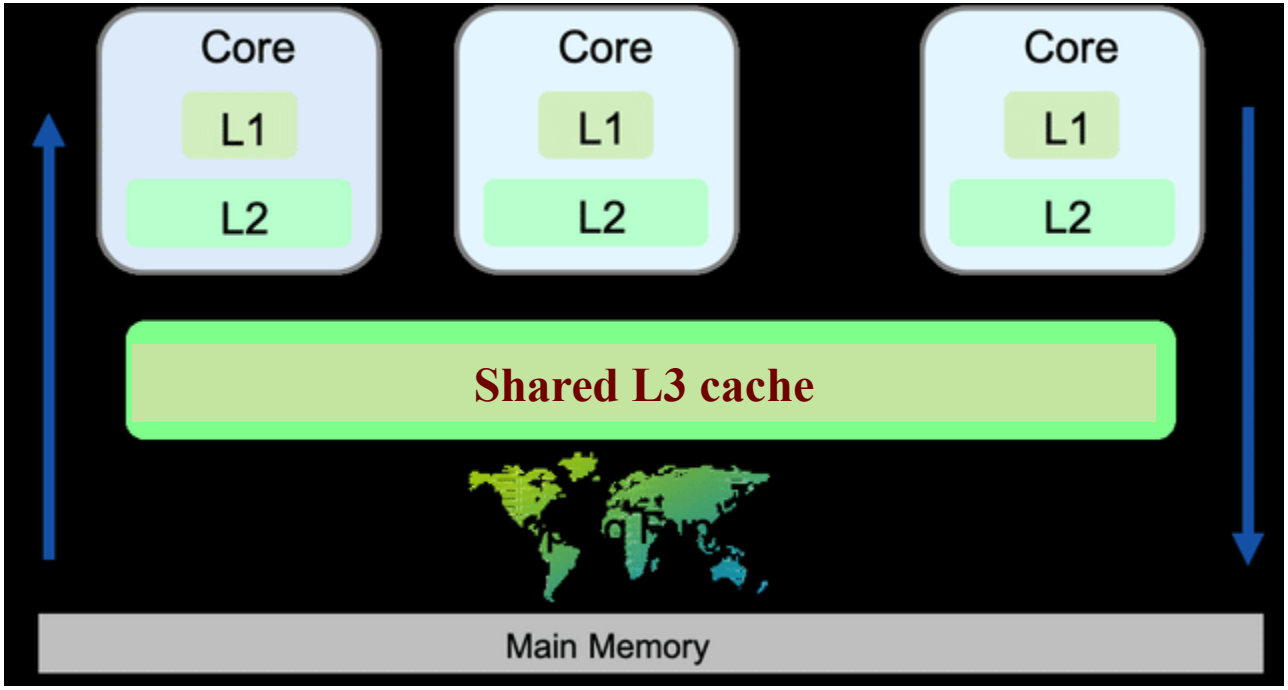






3) L3 cache

- The largest and slowest cache compared to L1 and L2.
- shared by multiple processor cores.
- With multicore processors, each core can have dedicated L1 and L2 cache, but they can share L3 cache.
- The memory size of this cache is up to 32MB, while AMD's revolutionary Ryzen 7 5800X3D CPUs come with 96MB L3 cache. Some server CPU L3 caches can exceed this, featuring up to 128MB.
- (twice as fast as the RAM)



Locality

- **Programs tends to reuse data and instructions near those they have used recently.**
- **The ability of cache memory to improve a computer's performance relies on the concept of locality of reference.**
- **Locality describes various situations that make a system more predictable.**
- **Cache memory takes advantage of these situations to create a pattern of memory access that it can rely upon.**

Locality of Reference

- **Locality of reference refers to a phenomenon in which a computer program tends to access same set of memory locations for a particular time period.**

- **In other words, Locality of Reference refers to the tendency of the computer program to access instructions whose addresses are near one another.**

Types of locality of reference

1) **Locality in space (Spatial Locality)**

This refers to accessing various data that are near each other (Arrays).

A neighbor of a recently referenced memory location is likely to be referenced.

For (int i==1;i<10;i++)

A[i]=a[i]+1 a[0] a[1] a[2]

Accessing Location x → location x+1 → location x+2 → array is stored in cache

2) **Locality in time (Temporal Locality)**

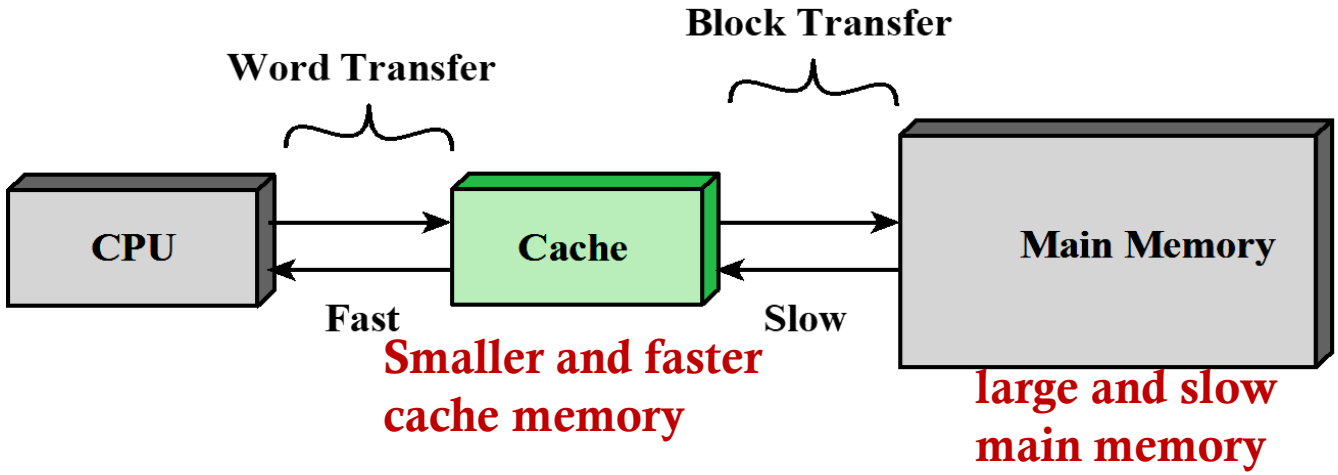
This is when the same data are accessed repeatedly in a short amount of time.

Recently referenced items are likely to be referenced in the near future

Location x after short time T refers to location x (Loop-structured and OOP Function call)

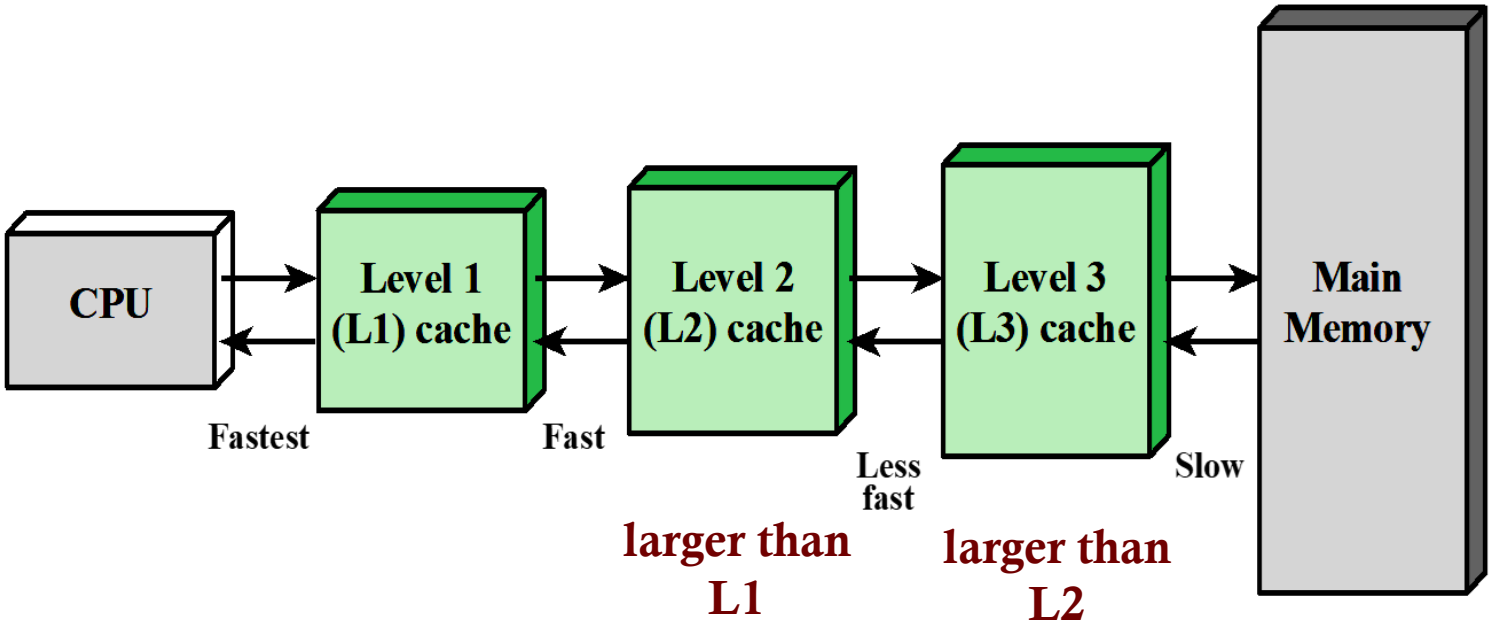
Note:

- **If there is no localize in code then, the cache would be as overhead**
- **Disadvantage of cache: more than one copy of data (consistency problem)**



Single cache

- **The cache** contains a copy of a portion of **main memory**.
- When **the processor** attempts to read **a byte or word** of memory, **a check** is made to determine if it is **in the cache**.
  - **If so**, **the byte or word** is **delivered** to **the processor**.
  - **If not**, **a block of main memory** **(Why?)** consisting of some fixed number of bytes, is read into **the cache** **and then** **the byte or word** is **delivered** to **the processor**.



Three-level cache organization

The **L2** cache is slower and typically larger than the **L1** cache, and the **L3** cache is slower and typically larger than the **L2** cache.

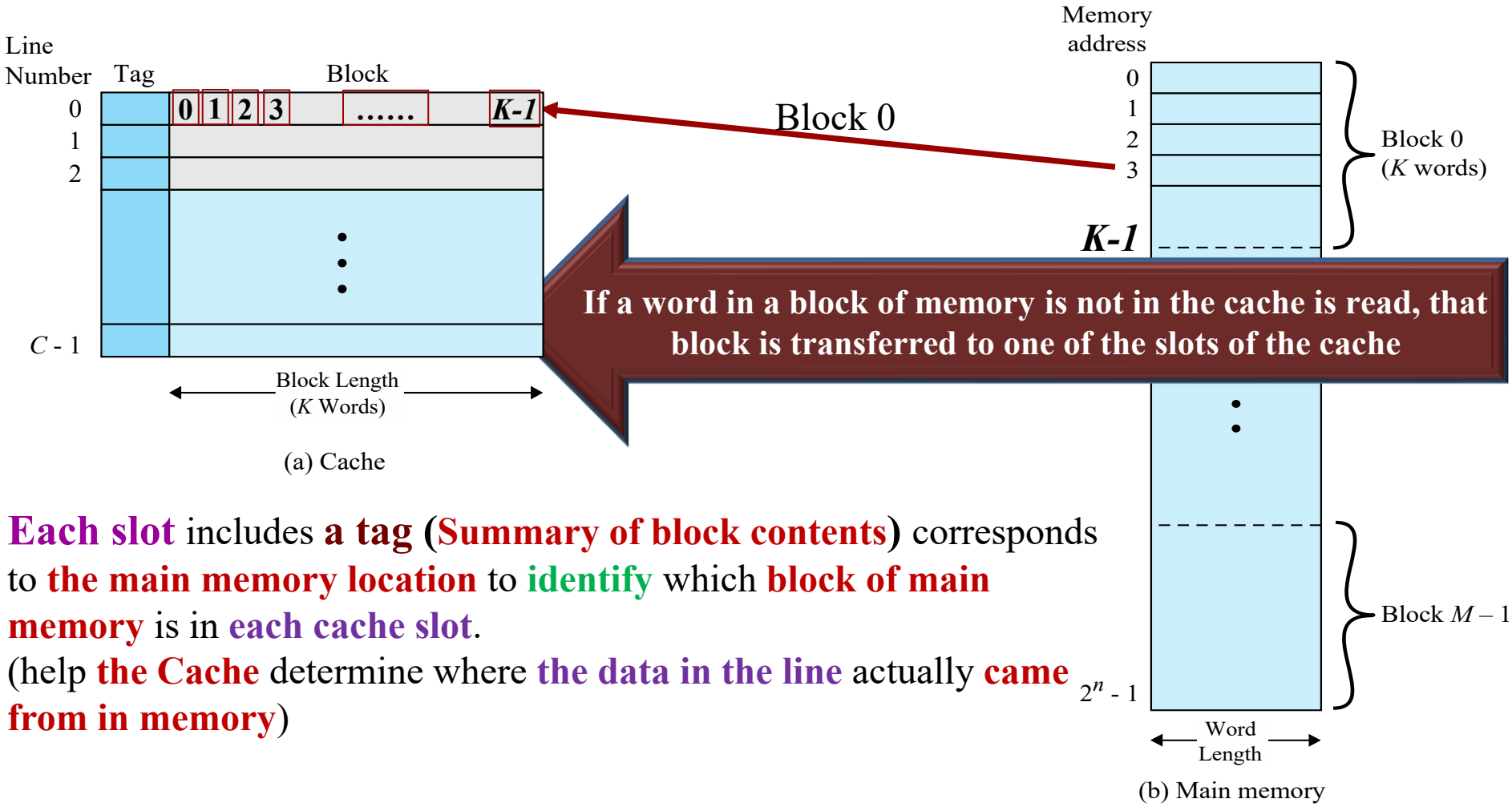
The design of cache memory

- **Cache memory is accessed based on the data content rather than the specific address or location.**
- **int x; → memory address**
- **x is stored in cache as variable**
- **Access to cache is not by address of x but by x**

Cache/Main-Memory Structure

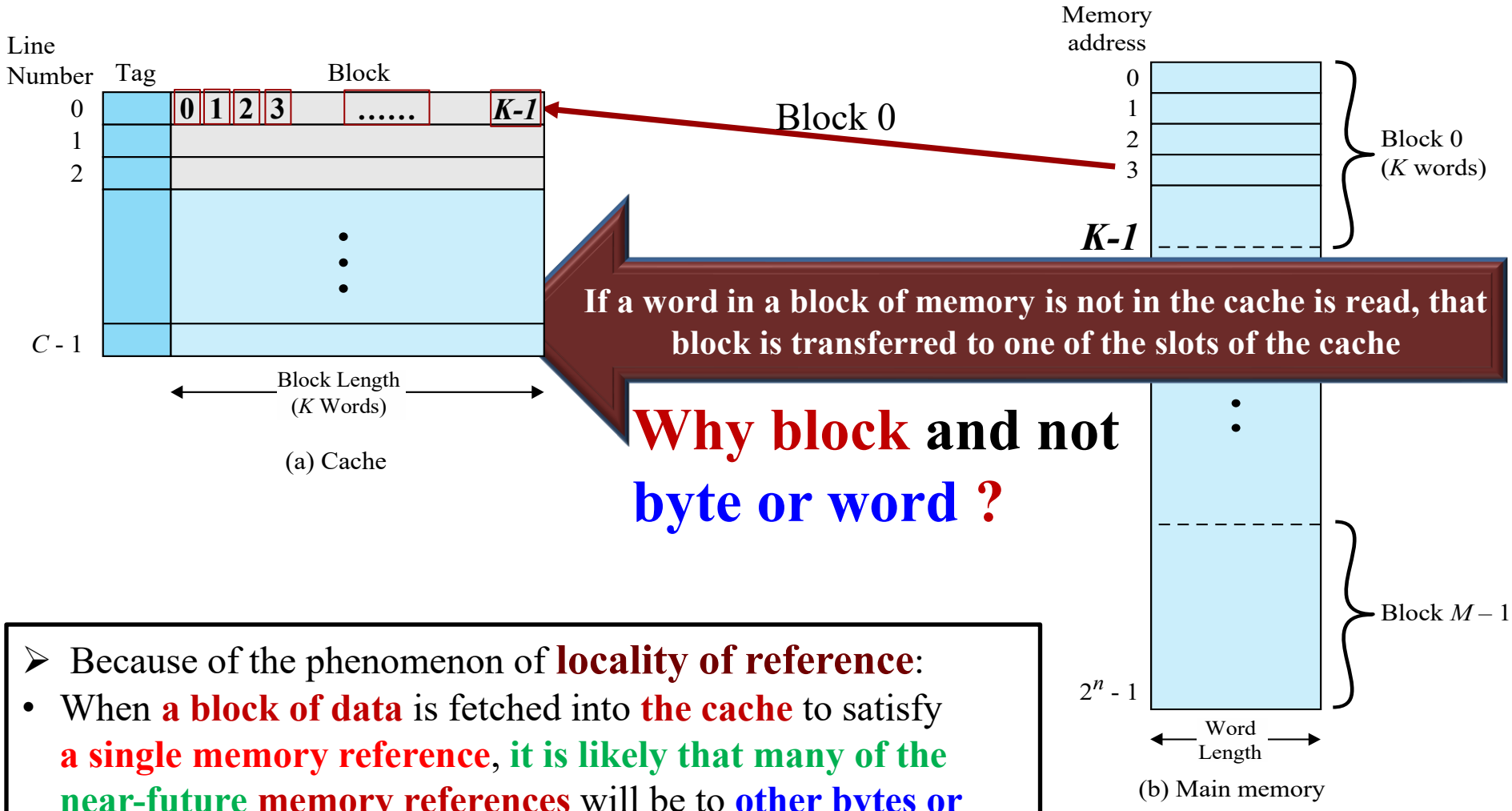
- *Cache consists* of **C slots** *(lines) of* ***K words each***.
- *the number of* **slots** *is* less than the number of main memory blocks (**C<<M**)

- ***n* address lines -> memory** consists of up to 2^n ***addressable words***
- **Memory** consist of a number of fixed-length **blocks** of ***K words each***.
- *there are* **M** $= 2^n/K$ ***blocks***



- **Each slot** includes **a tag (Summary of block contents)** corresponds to **the main memory location** to **identify** which **block of main memory** is in **each cache slot**.
- (help **the Cache** determine where **the data in the line** actually **came from in memory**)

Cache/Main-Memory Structure



- Because of the phenomenon of **locality of reference**:
- When **a block of data** is fetched into **the cache** to satisfy **a single memory reference**, **it is likely that many of the near-future memory references** will be to **other bytes or words in that block**.

(the tendency to access **byte or word whose address** are **near one another**.) -> **the references to memory** tend to be **confined/limited**

Each slot includes a tag that identifies which particular block is currently being stored

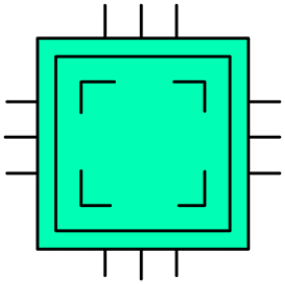
Suppose we have a 6-bit address and a 2-bit tag.

The tag 01 refers to the block of locations with the following addresses:

010000, 010001, 010010, 010011, 010100, 010101, 010110, 010111, 011000, 011001, 011010, 011011, 011100, 011101, 011110, 011111.

Cache Hit

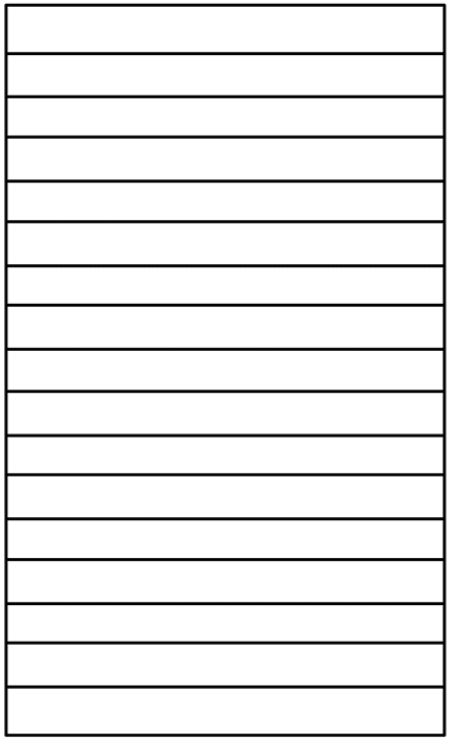
- When a program requests data from memory, the processor will first look in the cache.
- If the memory location matches one of the tags in a cache entry the result is a cache hit and the data is retrieved from the cache.



Cache

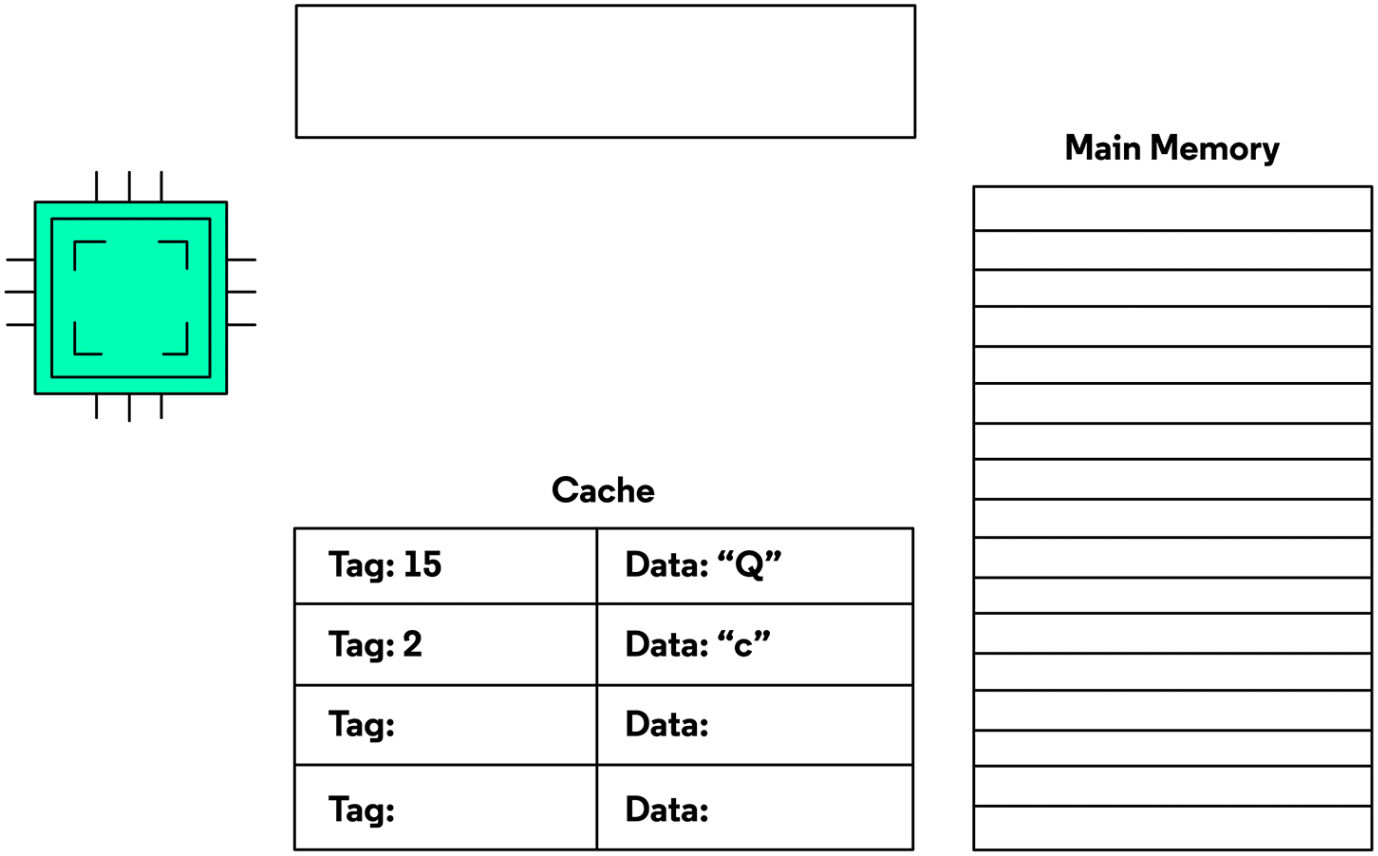
Tag: 15	Data: “Q”
Tag: 2	Data: “c”
Tag:	Data:
Tag:	Data:

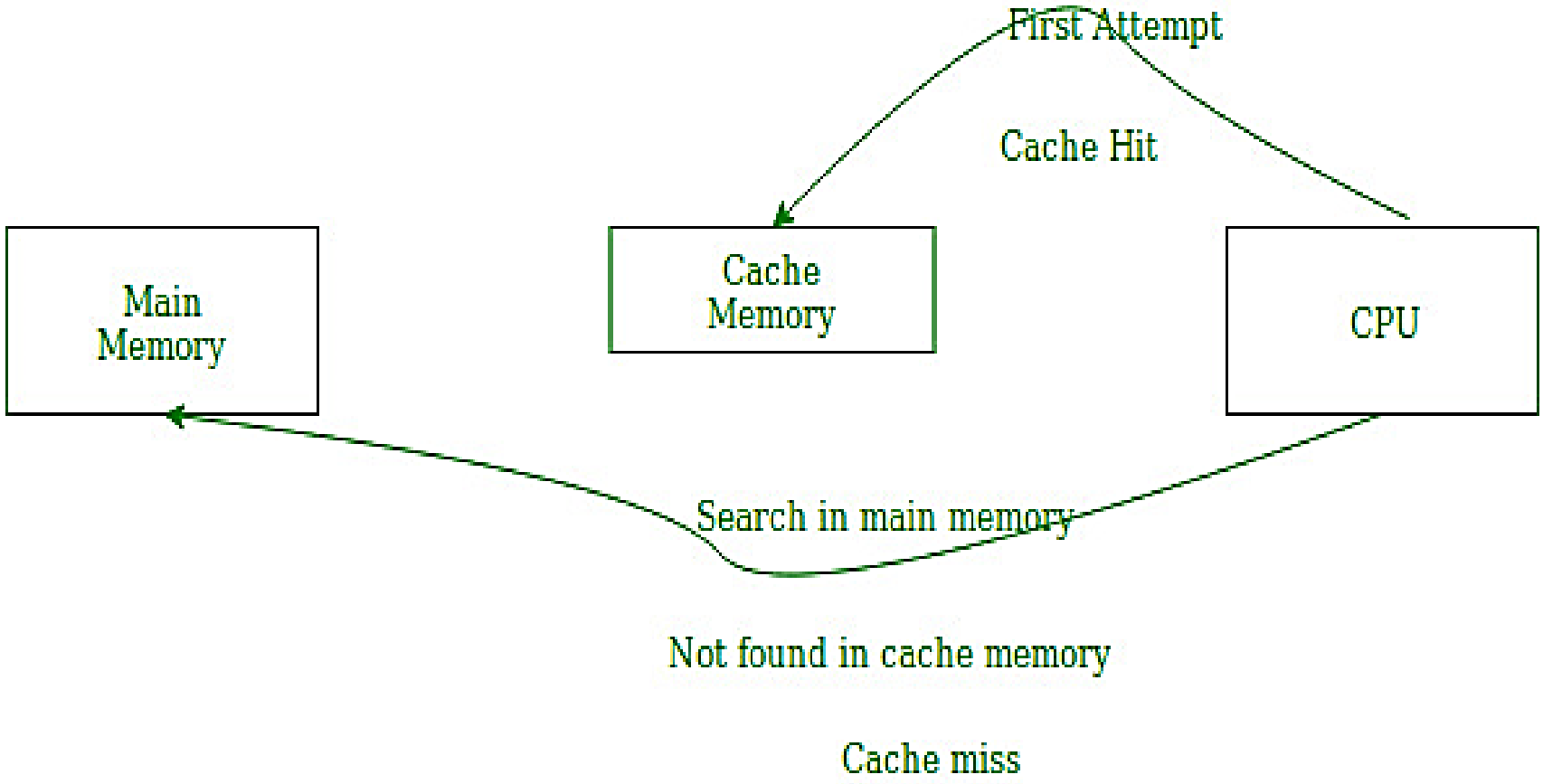
Main Memory

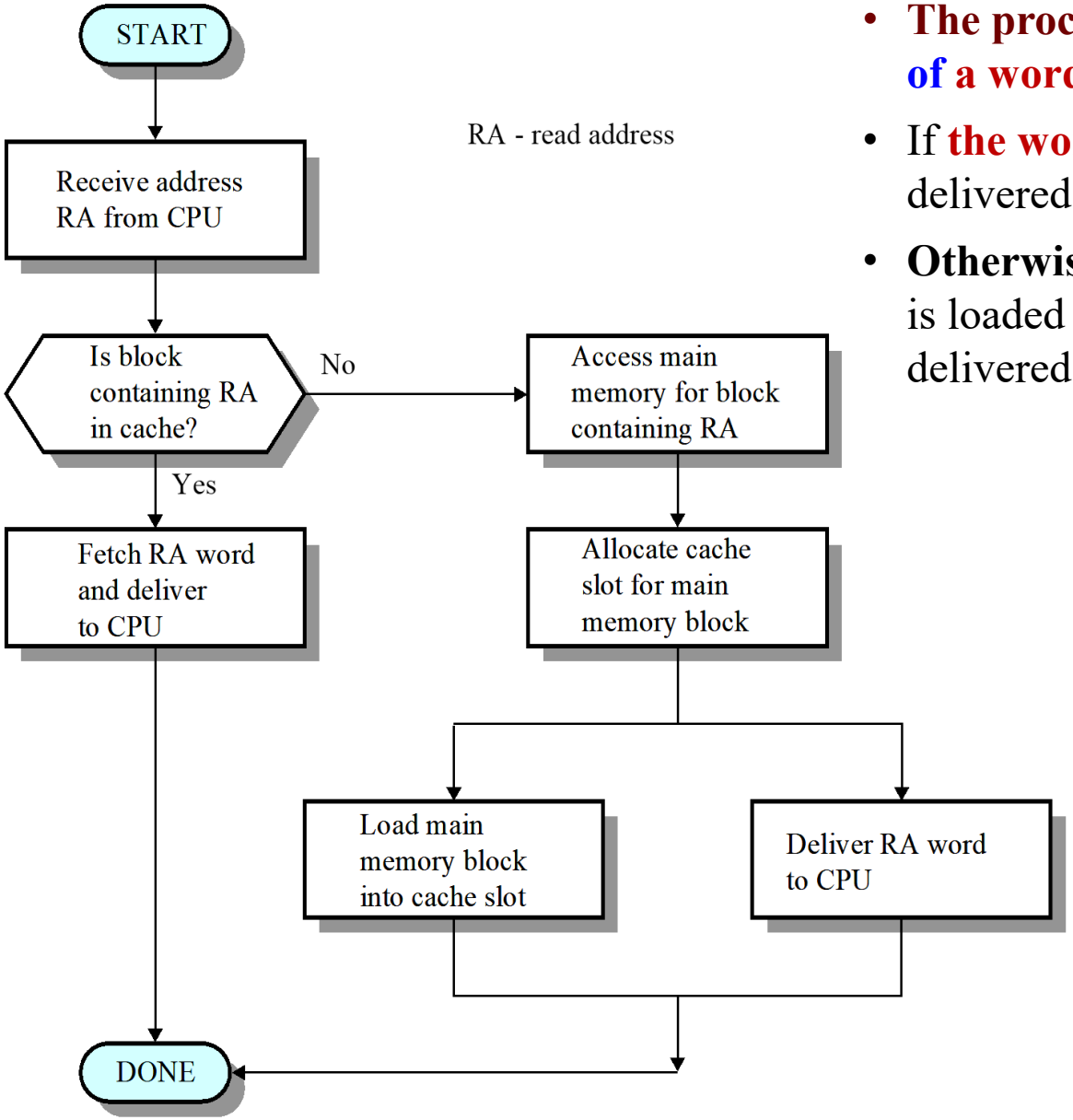


Cache miss

- A cache miss is when a processor does not find the data it needs in cache memory and must request it from the main memory.
- The main memory places the memory location and data in as an entry in the cache. The data is then retrieved by the processor from the cache.




First Attempt
Cache Hit
Main Memory
Cache Memory
CPU
Search in main memory
Not found in cache memory
Cache miss



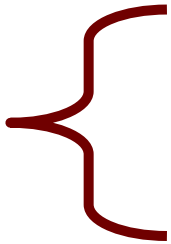
- **The processor** generates the address **RA of a word to be read**.
- If **the word** is contained in **the cache**, it is delivered to **the processor**.
- **Otherwise**, **the block** containing that **word** is loaded into **the cache** and the word is delivered to the processor.

Cache Read Operation

Cache Associative Memory

int x;

Summary of block contents
Search in parallel



Tag	Block

Search Time in tags almost 0

Average access time



T1 Access time T2

T1 <<T2

Average Access time= T1+T2 /2

x1 x2
Avg= x1+x2 /2
.5 * x1 + .5 *x2
Weight 1 * value 1 +
Weight 2 * value 2
Sum of weights =1
Assumption: all values are equiprobable

Weighted Average

Two cases:

Average Access time= Found in Cache + not Found in Cache

= Hit ratio * T1 + (1-Hit ratio) * (T2+T1)

Hit ratio = # Times found in Cache / Total # of Times

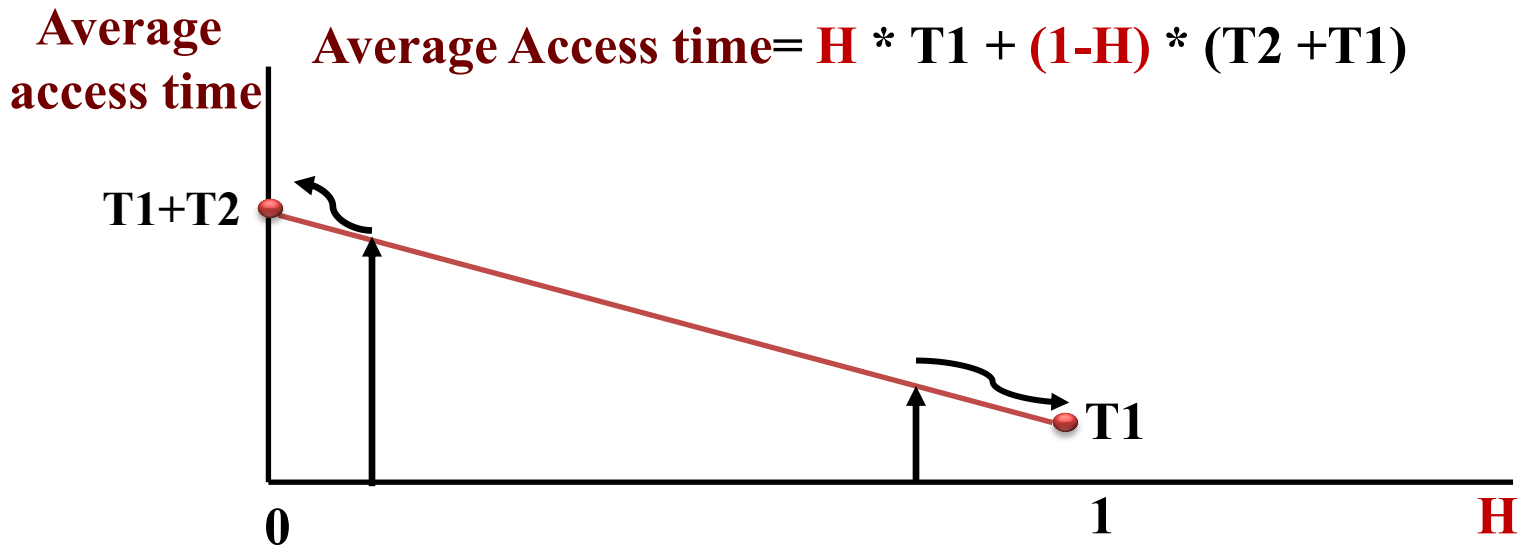
Hit Ratio (H) = hit / (Hit + Miss) = # of Hits/Total accesses

Miss Ratio = 1-Hit ratio (H)

Miss Ratio = miss / (hit + miss) = # of miss/total accesses

Average Access time= H * T1 + (1-H) * (T2 +T1)

Is The Performance enhanced using cache ???



Possible values of H

0 means code has no locality (Impossible)

1 means that cache is large enough as main memory that always data found in it (Impossible)

0 &1 is not realistic

If H=0 Average access time is T1+T2 (without cache time is only T2 !!!) → Worst

If H=1 Average access time is T1

So, for the cache to improve performance, H must be close to 1 How?

1- large cache size

2- writing a code with high level of locality

Why does cache improve performance ?

Because all programs, whether we like it or not, contain locality
→ away of 0 and according to size.

Virtual Memory

Processor	Cache	Main Memory	Secondary Storage
	T1	T2	T3

Access time

T1 <<T2 << T3

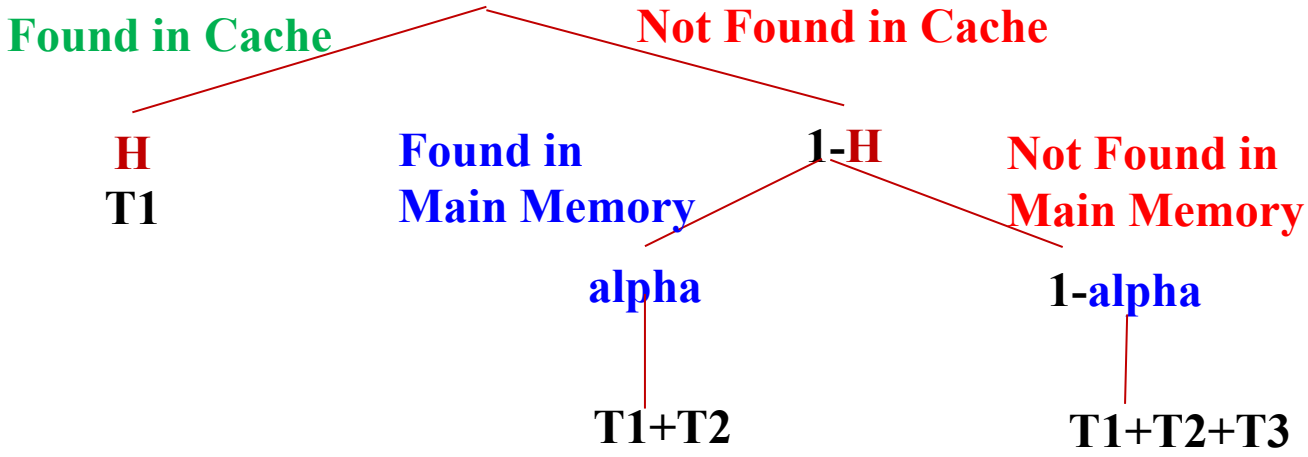
Average Access time= Found in Cache + not Found in Cache

Not Found in Cache (2 Cases) =

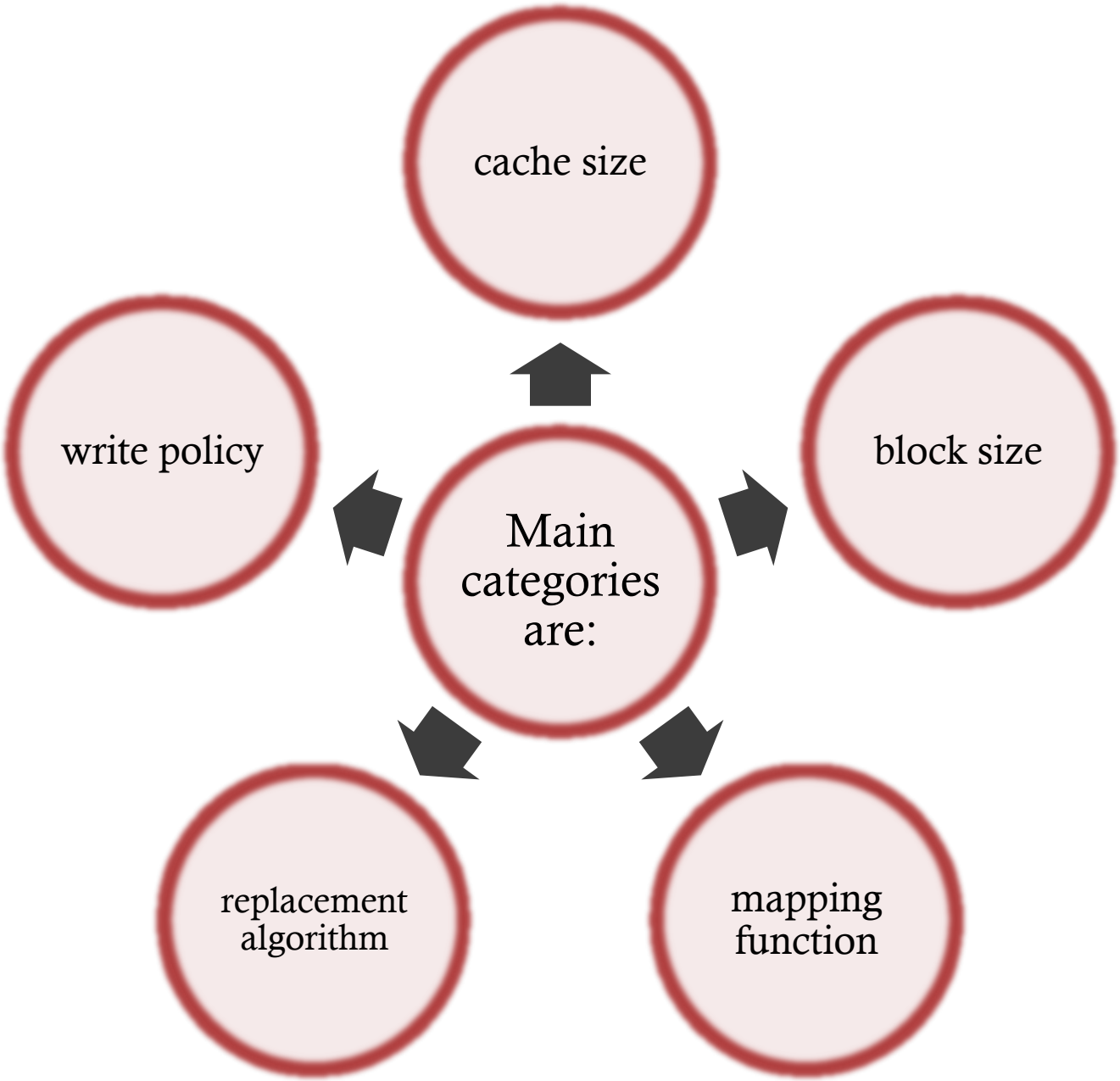
Found in Main Memory + not Found in Main Memory (from HDD)

- Assume that alpha is Hit ration of Main Memory

Average Access time = H * T1 + (1-H) (alpha* (T2 +T1) + (1-alpha) * (T3+T2 +T1))



Cache Design Issues



- **Cache size**
  - Small caches have a significant impact on performance
- **Block size**
  - **The unit of data** exchanged between **cache** and **main memory**
  - **Hit** means the information was found in **the cache**

- As the **block size increases**, **the Hit ratio** will **at first increase** because of **the principle of locality**.
- **The Hit ratio** will begin to **decrease**, as the **block size** becomes bigger, **the probability of using the newly fetched data** **becomes** **less than the probability of reusing the data that have to be moved out of the cache** to **make room** for **the new block**.

- Mapping function
  - Determines which cache location the block in main memory will occupy
- Replacement algorithm
  - Chooses which block to replace when a new block is to be loaded into the cache When all lines are occupied

1) Least Recently Used (LRU) Algorithm

– Replace a block that has been in the cache the longest with no references to it.

– Implementation: having a USE bit for each line

2) Least Frequently Used (LFU) Algorithm

- Replace that block with fewest references or hits.
- Implementation: associate a counter with each line and increment when used.

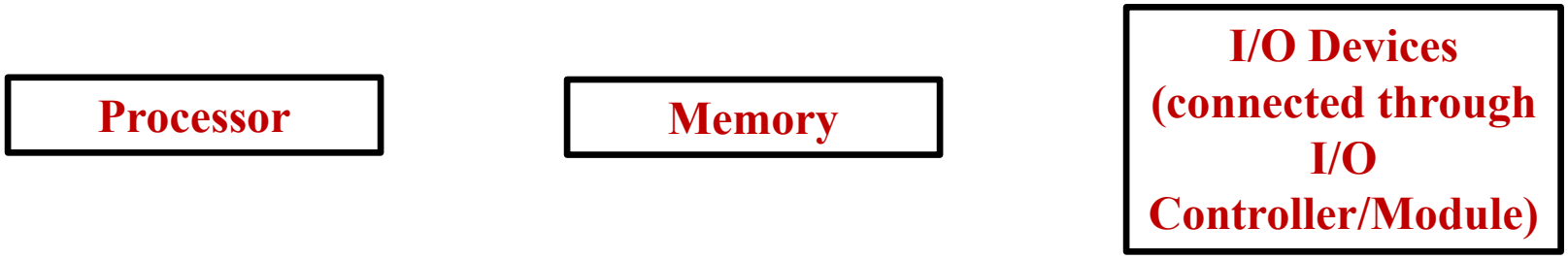
- Write policy (to handle consistency problem as there exist more than one copy of data)

Defines **how data is written to main memory** if the contents of a block in **the cache** are altered, **it is necessary to write it to main memory**.

- Can occur every time **block** is **updated (write Through)**
  - Writes **data** to **the cache** and **the main memory at the same time**. This policy is **easy to implement** in the architecture but **is not as efficient since every write to cache is a write to the slower main memory**.
- Can occur only when **block** is **replaced (write Back)**
  - Writes **data** to **the cache but** only writes **the data** to **the main memory when data** is about to **be replaced** in **the cache**.
    - **Minimizes** memory operations
    - Leaves memory in an **obsolete state**(As both cache and main memory have different data)

I/O Communication Techniques

- When **the processor** encounters an instruction relating to **I/O**, it executes that instruction by issuing **a Read command** to **the appropriate I/O module**.
- **I/O data transfer techniques** are used to **transfer data** **from I/O devices to memory** and vice versa.



I/O Communication Techniques

Three techniques are possible for I/O operations:

Programmed I/O (Polling)	**Interrupt-Driven I/O**	**Direct Memory Access (DMA)**

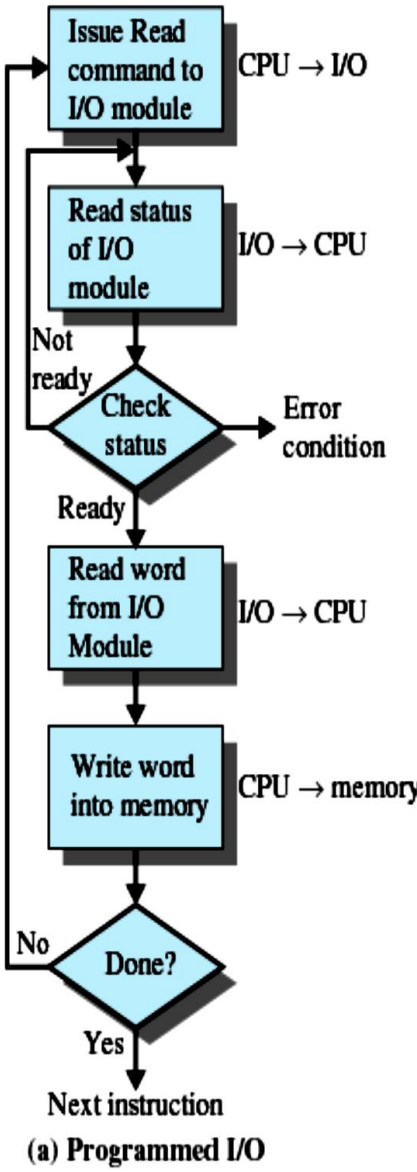
Programmed I/O (Polling)

- In this technique, **the I/O device doesn't have direct memory access**.

1) **Processor** sends '**Read**' command to the **I/O module/Controller**
2) **Processor periodically checks the status of the I/O module.**
3) **If the status** is ready/done (**data in buffer**), the **Processor** read the **word** from the **I/O module buffer** and **writes** it into the **memory**.

➢ If the operation was done successfully, **the processor** goes on to the **next instruction**.

The I/O module performs **the requested action** then sets the appropriate bits in the I/O status register. (**it does not interrupt the processor**)



(a) Programmed I/O

Programmed I/O

- With programmed I/O the **performance** level of the entire system is severely **degraded** (**Waste of processor time**) Why?

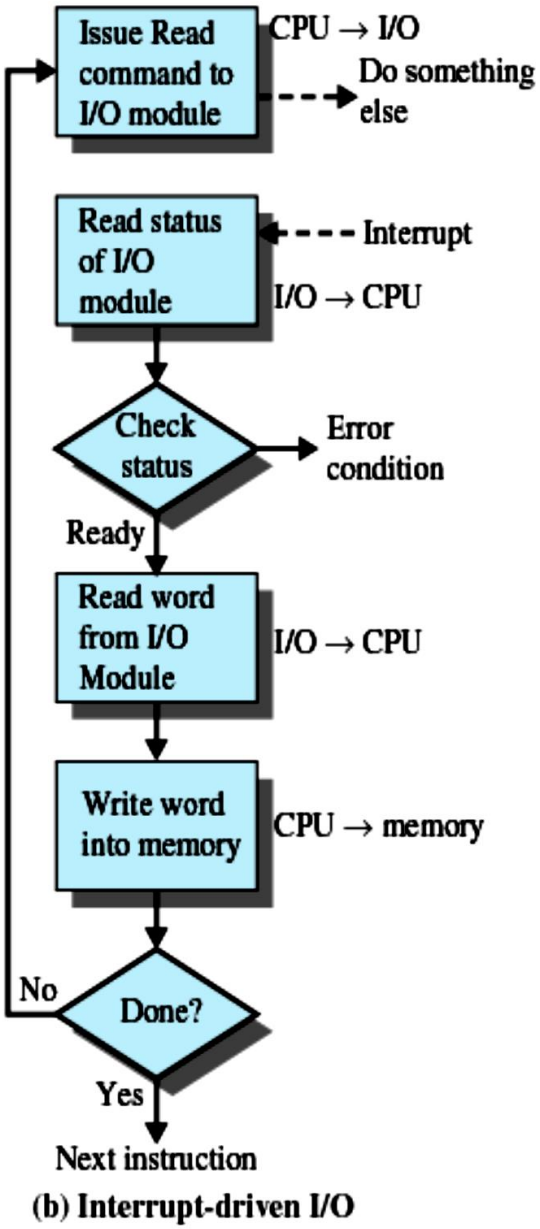
1) **The processor** has to **wait a long time for the I/O module of concern to be ready** for reception or transmission of more data.

2) **The processor**, while waiting, must **repeatedly interrogate the status of the I/O module**.

Interrupt-Driven I/O

1) **Processor** sends the '**Read**' command to the **I/O module** and then goes on to **do some useful work**.
2) **When I/O module** is **ready/done** (**data in buffer**), it sends **an interrupt signal** to **CPU**.
   - When the **Processor** receives the **interrupt signal**, it checks the **status**.
3) **If the status** is **ready**, then the **CPU read** the **word** from the **I/O module buffer** and **writes** the word into the **memory**.

➢ If the operation was done successfully, **the processor** goes on to the **next instruction**.

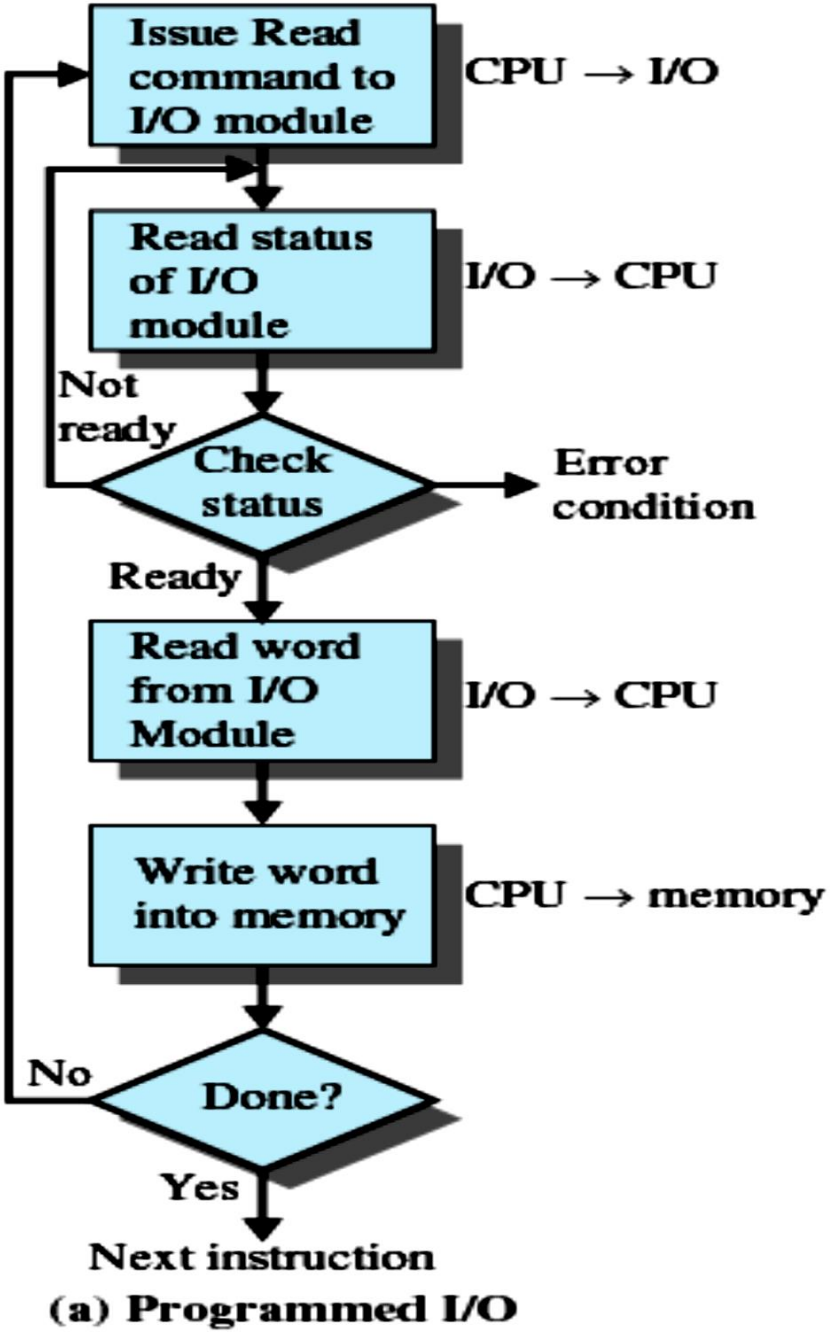


(b) Interrupt-driven I/O

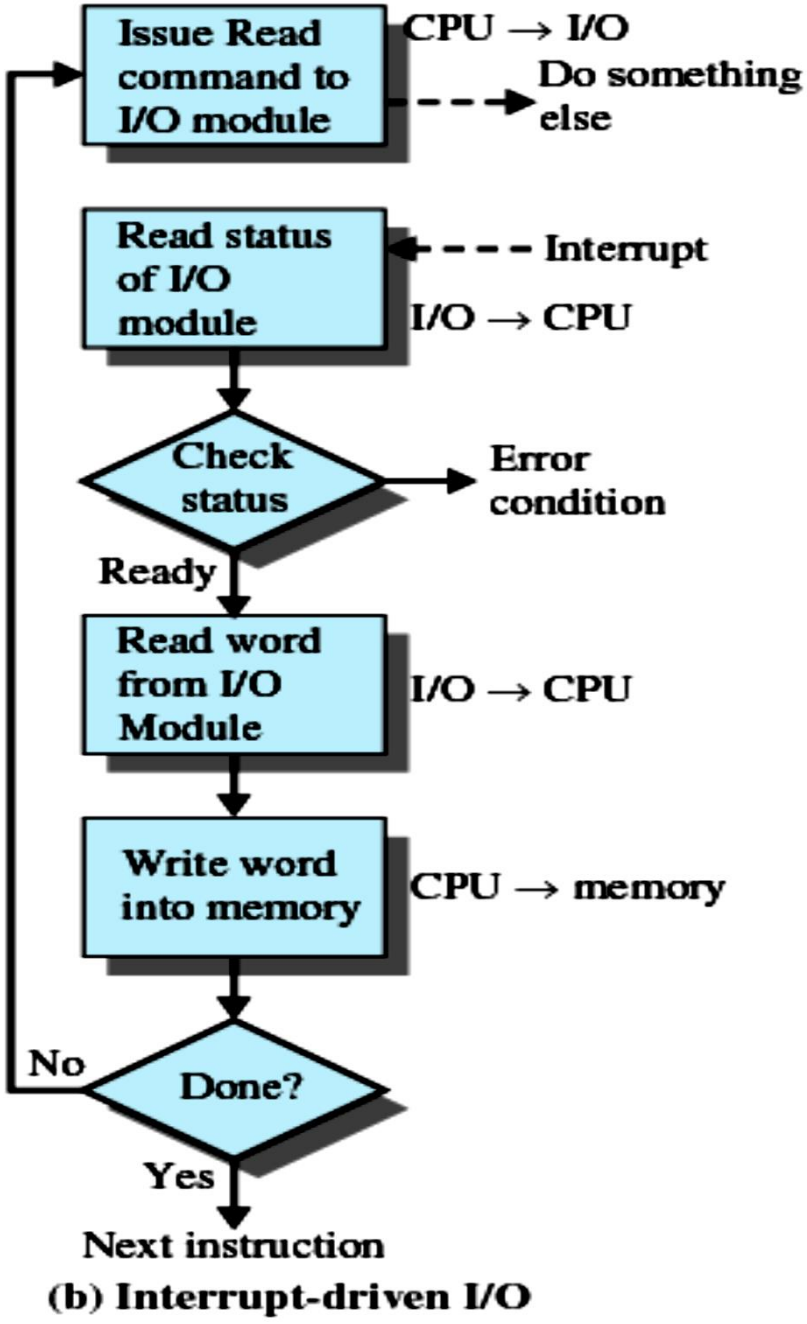
Programmed I/O and Interrupt-Driven I/O Drawbacks

1) **Waste of processor time**
   - **Because every word read or written passes through the processor**
   - **The processor** is **tied up in managing an I/O transfer**
     - A number of instructions must be executed for each I/O transfer

2) **Transfer rate** is limited by the **speed with which the processor can test and service a device.**



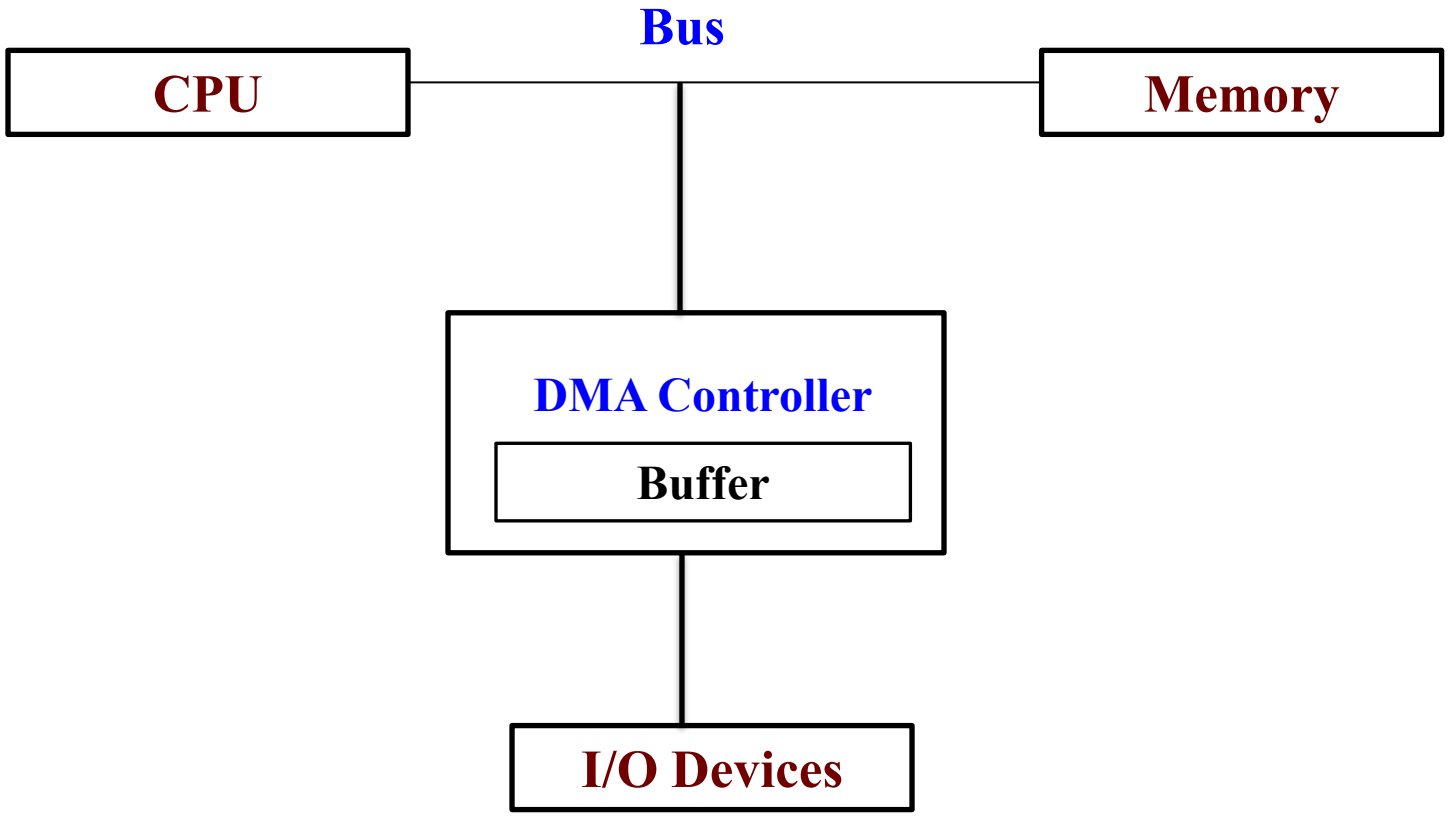
(a) Programmed I/O



(b) Interrupt-driven I/O

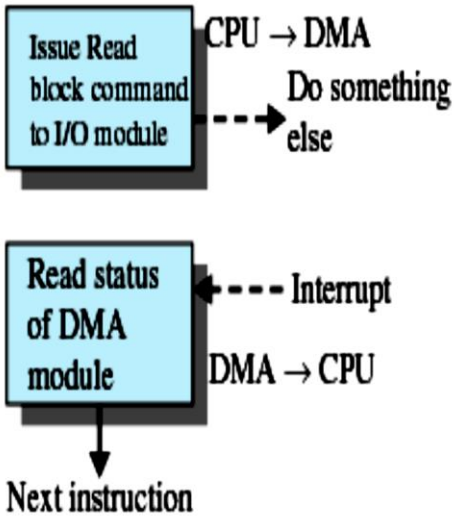
Direct Memory Access (DMA)

- **I/O Controller → DMA Controller**
- **The DMA function** can be performed by **a separate module** on **the system bus** or it can be **incorporated into an I/O module**.



Direct Memory Access (DMA)

1) **The processor wishes to read or write data, it issues a command to the DMA module containing:**
   - Whether a read or write is requested
   - The address of the I/O device involved
   - The starting location in memory to read/write
   - The number of words to be read/written



- ➢ It has delegated this **I/O operation** to the **DMA module** to manage the transfer directly between **the I/O device** and **memory**.
- ➢ **The processor** then **continues with other work**.

2) **DMA module** transfers **the entire block of data** directly to and from **memory without going through the processor**.

3) **When** the transfer is complete, the **DMA module** sends **an interrupt signal** to **processor**.

Thus, **the processor** is involved only at the **beginning** and **end** of the transfer

I/O Controller → I/O Processor

Merits and Demerits of Direct Memory Access

- **The processor** executes more **slowly** during a **DMA transfer when processor access** to the **bus** is **required**.
  - The **DMA module** needs to take control of the **bus to transfer data to and from memory**.
  - **Because of this competition for bus usage**, there may be times when the **processor** needs **the bus** and **must wait** for the **DMA module**.
- ✓ **For** a **multiple-word I/O transfer**, **DMA** is far **more efficient** than **interrupt-driven** or **programmed I/O**.


Any Questions ?
Thank you